

Projeto e Análise de Algoritmos

Ordenação

Thiago Pinheiro de Araújo
EMAp/FGV
2026.2

Apresentar **algoritmos de ordenação**,
discutir as características de cada um,
analisar a complexidade, e exibir exemplos
de implementação.

O problema

- Dada uma sequência de valores, escreva um algoritmo capaz de retornar a sequência ordenada.

```
int v[] = {8, 11, 2, 5, 10, 16, 7, 15, 1, 4};
```

- Exceto quando especificado de outra forma, assuma que o tipo dos valores são números inteiros.
- Utilizaremos o vetor como estrutura de dados, no entanto os algoritmos apresentados podem ser implementados utilizando outras estruturas
 - Ex: listas encadeadas.

O problema

- Um algoritmo de ordenação é considerado estável se não alterar a posição relativa de elementos com o mesmo valor
 - Ex: considere um algoritmo de ordenação sobre valores decimais que apenas ordena a parte inteira

8.7	11.4	2.1	5.5	11.3	2.7	7.1	5.3	9.0	4.8
-----	------	-----	-----	------	-----	-----	-----	-----	-----

2.1	2.7	4.8	5.5	5.3	7.1	8.7	9	11.4	11.3
-----	-----	-----	-----	-----	-----	-----	---	------	------

Bubble sort

Bubble sort

- O algoritmo **bubble sort** (ordenação por flutuação) é uma das soluções mais simples para o problema de ordenação.
- A solução consiste em inverter (trocar) valores de posições adjacentes sempre que $v[i + 1] < v[i]$.
- Essa operação é executada para cada posição $0 \leq i < n - 1$ ao percorrer a sequência.
- Observe que ao percorrer a sequência $j = n - 1$ vezes executando esse procedimento atingimos a sequência ordenada.

Bubble sort

- **Exemplo:** considere a seguinte sequência

8	11	2	5	1
---	----	---	---	---

$$n = 5$$

$$j = n - 1 = 4$$

- Podemos ordená-la utilizando bubble sort da seguinte forma:

j=0

8	11	2	5	1
8	11	2	5	1
8	2	11	5	1
8	2	5	11	1

j=1

8	2	5	1	11
2	8	5	1	11
2	5	8	1	11
2	5	1	8	11

j=2

2	5	1	8	11
2	5	1	8	11
2	1	5	8	11
2	1	5	8	11

j=3

2	1	5	8	11
1	2	5	8	11
1	2	5	8	11
1	2	5	8	11

- Exemplo de implementação:

```
#define swap(v, i, j) { int temp = v[i]; v[i] = v[j]; v[j] = temp; }
```

```
void bubbleSort(int v[], int n) {  
    for (int j = 0; j < n - 1; j++) {  
        for (int i = 0; i < n - 1; i++) {  
            if (v[i] > v[i + 1]) {  
                swap(v, i, i + 1);  
            }  
        }  
    }  
}
```

Bubble sort

- Vamos **avaliar o desempenho** através do número de comparações necessárias para ordenar a sequência.
- O procedimento é executado j vezes, e $j = n - 1$.
- A cada execução do procedimento realizamos $n - 1$ comparações.
- Portanto são realizadas $(n - 1) \cdot (n - 1)$ comparações no total
 - Observe que esse número de comparações independe do estado inicial.
- Logo o algoritmo apresenta complexidade $\Theta(n^2)$ para o melhor e o pior caso.

Bubble sort

- Podemos otimizar o algoritmo para o melhor caso interrompendo a execução quando o vetor estiver ordenado
 - Esse estado pode ser identificado quando ocorrer uma execução do procedimento que não efetue uma troca de valores.

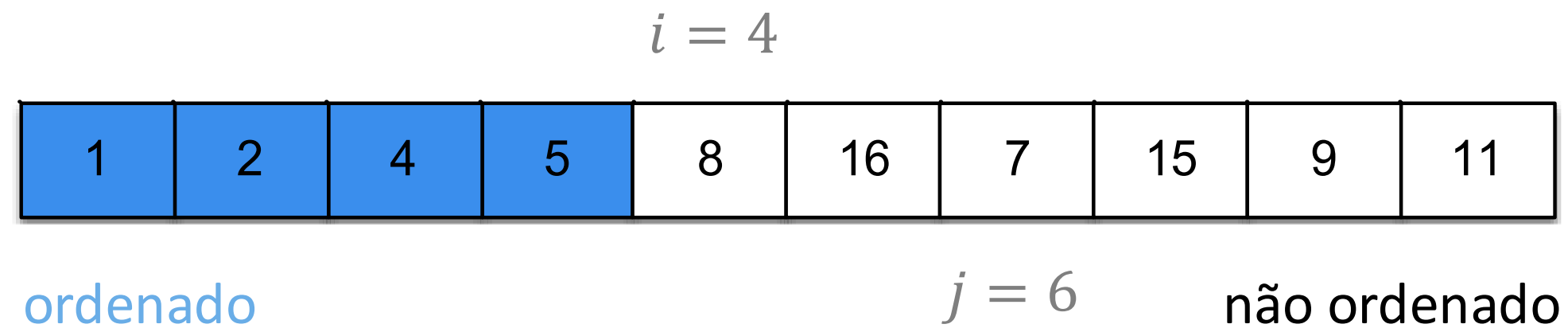
```
void bubbleSortOptimized(int v[], int n) {  
    for (int j = 0; j < n - 1; j++) {  
        bool swapped = false;  
        for (int i = 0; i < n - 1; i++) {  
            if (v[i] > v[i + 1]) {  
                swap(v[i], v[i + 1]);  
                swapped = true;  
            }  
        }  
        if (!swapped) { break; }  
    }  
}
```

- A complexidade do melhor caso passa a ser $\theta(n)$.

Selection sort

Selection sort

- A **ordenação por seleção** (*selection sort*) busca para cada posição o *i-ésimo* valor que deveria residir nela caso a sequência estivesse ordenada.



- Dada uma posição i , busca-se encontrar a posição j com o menor valor dentre as posições $[i..n - 1]$
 - A cada varredura o valor da posição i é invertido com o valor da posição j , tornando a sequência $[0..i - 1]$ ordenada.

Ordenação

Selection sort

- **Exemplo:** considere a mesma sequência

8	11	2	5	1
---	----	---	---	---

 $n = 5$

- Podemos ordená-la utilizando o algoritmo *selection sort* da seguinte forma:

$i = 0$	<table><tr><td>8</td><td>11</td><td>2</td><td>5</td><td>1</td></tr></table>	8	11	2	5	1	$\min\text{ }inx = 4$
8	11	2	5	1			
$i = 1$	<table><tr><td>1</td><td>11</td><td>2</td><td>5</td><td>8</td></tr></table>	1	11	2	5	8	$\min\text{ }inx = 2$
1	11	2	5	8			
$i = 2$	<table><tr><td>1</td><td>2</td><td>11</td><td>5</td><td>8</td></tr></table>	1	2	11	5	8	$\min\text{ }inx = 3$
1	2	11	5	8			
$i = 3$	<table><tr><td>1</td><td>2</td><td>5</td><td>11</td><td>8</td></tr></table>	1	2	5	11	8	$\min\text{ }inx = 4$
1	2	5	11	8			
	<table><tr><td>1</td><td>2</td><td>2</td><td>8</td><td>11</td></tr></table>	1	2	2	8	11	
1	2	2	8	11			

- Exemplo de implementação:

```
void selectionSort(int v[], int n) {  
    for (int i = 0; i < n - 1; i++) {  
        int minInx = i;  
        for (int j = i + 1; j < n; j++) {  
            if (v[j] < v[minInx]) {  
                minInx = j;  
            }  
        }  
        swap(v, i, minInx);  
    }  
}
```

- Podemos analisar a corretude do algoritmo analisando para cada passo i as seguintes invariantes:
 - A sequência de valores entre $[0..n - 1]$ é uma permutação da sequência de entrada.
 - A sequência $[0..i - 1]$ está ordenada.
 - O valor da posição $i - 1$ é menor ou igual à qualquer valor de $[i..n - 1]$.

$$i = 4$$

1	2	4	5	8	16	7	15	9	11
---	---	---	---	---	----	---	----	---	----

ordenado

não ordenado

- Podemos **avaliar o desempenho** calculando o número de comparações necessárias para ordenar uma sequência de tamanho n no pior caso:

Iteração (i)	Número de comparações
0	$n-1$
1	$n-2$
2	$n-3$
...	...
$n-3$	2
$n-2$	1
$n-1$	0

- O número de comparações total consiste na soma da segunda coluna da tabela ao lado:

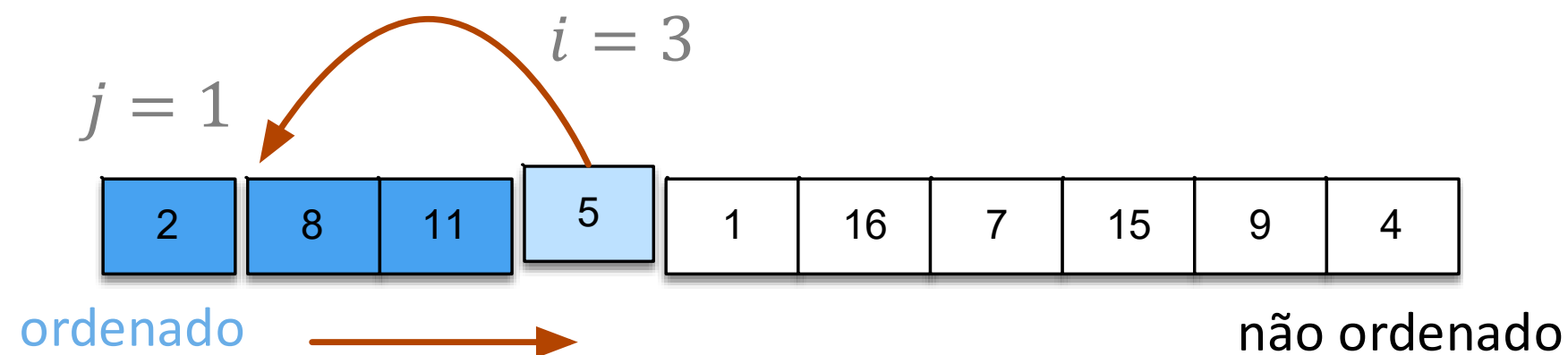
$$(n-1) + (n-2) + \dots + 2 + 1 + 0 = \sum_{i=0}^{n-1} i = \frac{n(n-1)}{2}$$

- Logo o algoritmo apresenta complexidade $\Theta(n^2)$ para o pior caso
 - Esta também é a complexidade no melhor caso.

Insertion sort

Insertion sort

- A **ordenação por inserção** (*insertion sort*) consiste em percorrer a sequência re-inserindo cada elemento de forma ordenada
 - Trivia: esse algoritmo costuma ser associado à ordenação de cartas de baralho em uma mão.

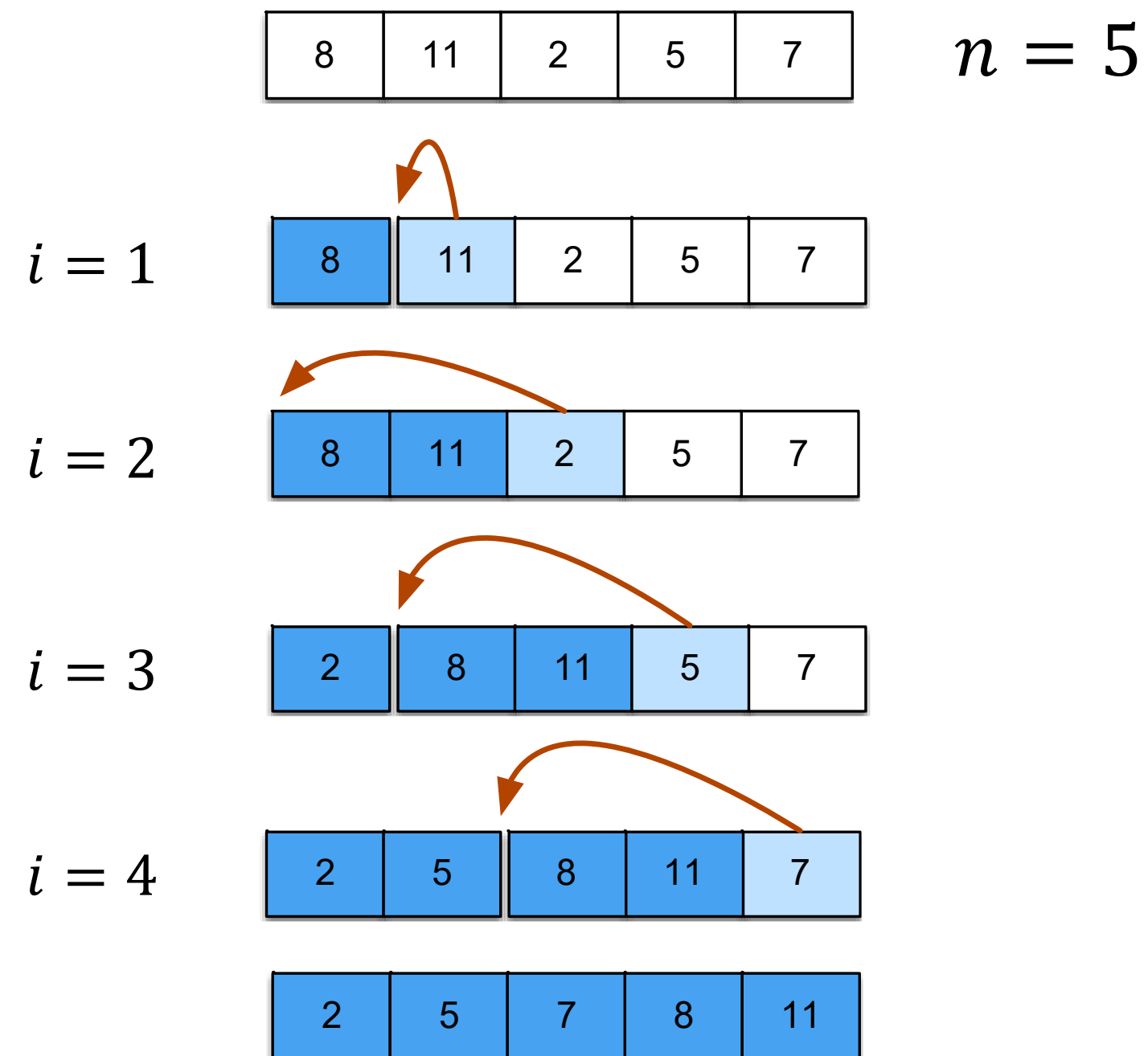


- Dada uma posição i , busca-se a posição j entre $[0..i - 1]$ percorrendo a sequência em ordem decrescente (a partir da posição $i - 1$) até $v[i] > v[j]$ ou atingir o início da lista
 - A cada passo o valor de $v[j + 1]$ é atualizado com o valor de $v[j]$.
 - Ao término da busca o valor original da posição i é trocado com o valor da posição $j + 1$, tornando a sequência $[0..i]$ ordenada.

Ordenação

Insertion sort

- **Exemplo:** vamos ordenar a sequência a seguir utilizando *insertion sort*:



- Exemplo de implementação:

```
void insertionSort(int v[], int n) {  
    for (int i = 1; i < n; i++) {  
        int currentValue = v[i];  
        int j;  
        for (j = i - 1; j >= 0 && v[j] > currentValue; j--) {  
            v[j + 1] = v[j];  
        }  
        v[j + 1] = currentValue;  
    }  
}
```

- Podemos analisar a corretude do algoritmo analisando para cada passo i as seguintes invariantes:
 - A sequência de valores entre $[0..n - 1]$ é uma permutação da sequência de entrada.
 - A sequência $[0..i - 1]$ está ordenada.

$$i = 4$$

1	2	4	5	8	16	7	15	9	11
---	---	---	---	---	----	---	----	---	----

ordenado

não ordenado

- Podemos **avaliar o desempenho** calculando o número de comparações necessárias para ordenar uma sequência de tamanho n no pior caso:

Iteração (i)	Número de comparações
1	1
2	2
3	3
...	...
n-3	n-3
n-2	n-2
n-1	n-1

- O número de comparações total consiste na soma da segunda coluna da tabela ao lado:

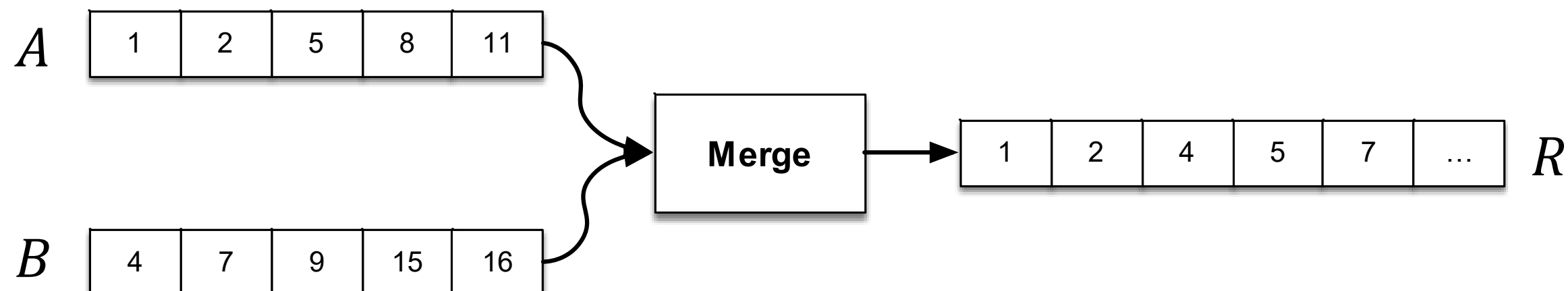
$$1 + 2 + 3 + \dots + (n - 2) + (n - 1) = \sum_{i=1}^{n-1} i = \frac{n(n - 1)}{2}$$

- Logo o algoritmo apresenta complexidade $\Theta(n^2)$ para o pior caso
 - No entanto, observe que a complexidade é $\Theta(n)$ para o melhor caso, quando a sequência encontra-se ordenada.

Mergesort

Mergesort

- O algoritmo **Mergesort** consiste em dividir a sequência em duas partes, executar chamadas recursivas para cada sub-sequência, e juntá-las (merge) de forma ordenada.
- Esse algoritmo depende de um algoritmo auxiliar de intercalação (merge)
 - Problema: dadas duas sequências A e B ordenadas produza uma sequência R ordenada contendo os elementos de ambas as sequências.



Mergesort

- Exemplo de implementação do algoritmo de intercalação:

```
void merge(int v[], int startA, int startB, int endB)
{
    int r[endB - startA];
    int aInx = startA;
    int bInx = startB;
    int rInx = 0;
    while (aInx < startB && bInx < endB) {
        r[rInx++] = v[aInx] <= v[bInx] ? v[aInx++] : v[bInx++];
    }
    while (aInx < startB) { r[rInx++] = v[aInx++]; }
    while (bInx < endB) { r[rInx++] = v[bInx++]; }
    for (aInx = startA; aInx < endB; ++aInx) {
        v[aInx] = r[aInx - startA];
    }
}
```

- Análise do desempenho: dado que cada sequência de entrada possui n elementos, são executadas $2n$ operações, portanto a complexidade é $\Theta(n)$

- O algoritmo Mergesort consiste nas seguintes etapas:

1. Divide-se a sequência em 2 partes A e B com aproximadamente o mesmo tamanho.

8	11	2	5	1	16	7	15	9	4
---	----	---	---	---	----	---	----	---	---

2. Executa uma chamada recursiva para ordenar a sequência A .

8	11	2	5	1	16	7	15	9	4
---	----	---	---	---	----	---	----	---	---

3. Executa uma chamada recursiva para ordenar a sequência B .

8	11	2	5	1	16	7	15	9	4
---	----	---	---	---	----	---	----	---	---

2	5	1
---	---	---

15	9	4
----	---	---

- O algoritmo Mergesort consiste nas seguintes etapas:
 1. Divide-se a sequência em 2 partes A e B com aproximadamente o mesmo tamanho.

1	2	4	5	7	8	9	11	15	16
---	---	---	---	---	---	---	----	----	----

2. Executa uma chamada recursiva para ordenar a sequência A .

1	2	5	8	11	4	7	9	15	16
---	---	---	---	----	---	---	---	----	----

3. Executa uma chamada recursiva para ordenar a sequência B .

8	11	1	2	5	7	16	4	9	15
---	----	---	---	---	---	----	---	---	----

4. Executa o algoritmo de intercalação para A e B .

5. Retorna a sequência R resultante.

2	5	1
---	---	---

9	15	4
---	----	---

Mergesort

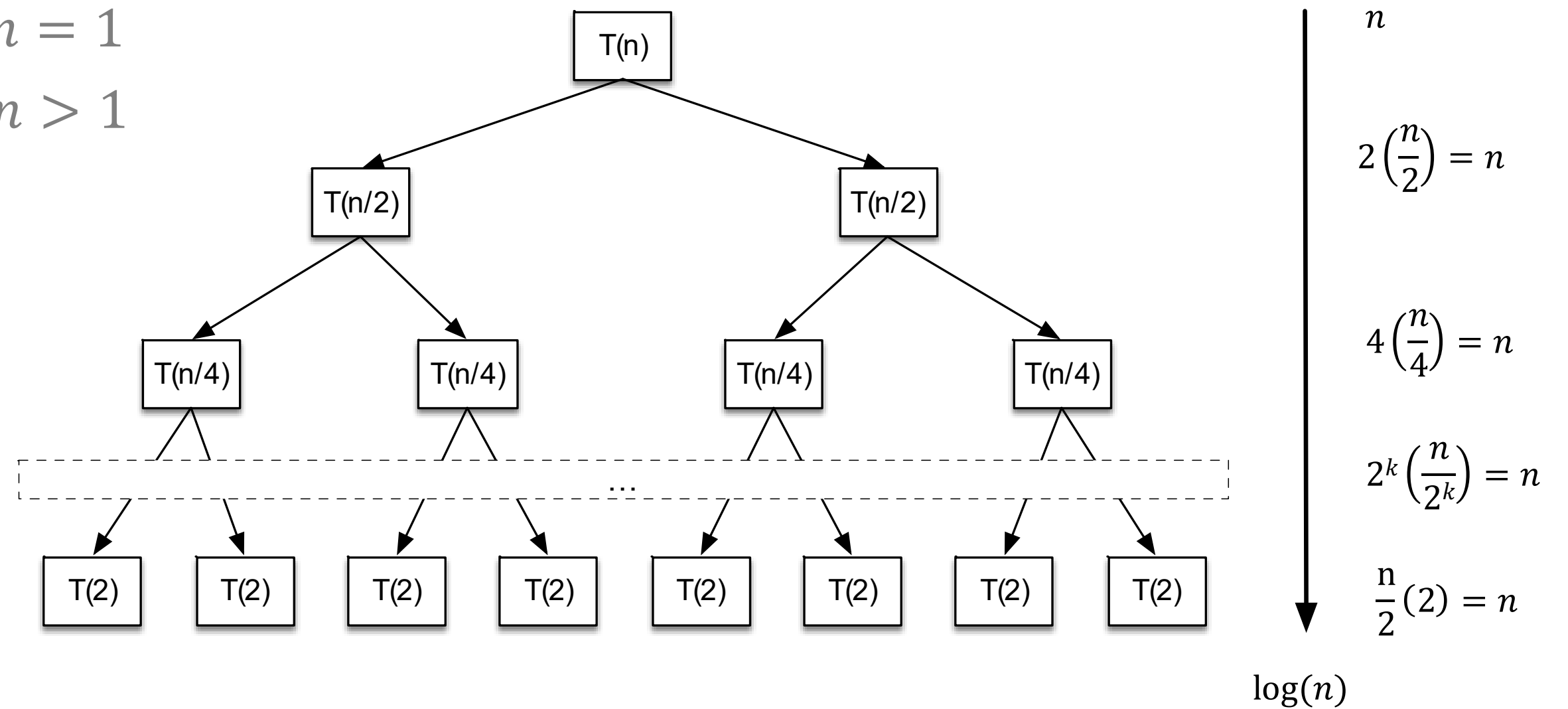
- Exemplo de implementação:

```
void mergeSort(int v[], int startInx, int endInx) {  
    if (startInx < endInx - 1) {  
        int midInx = (startInx + endInx) / 2;  
        mergeSort(v, startInx, midInx);  
        mergeSort(v, midInx, endInx);  
        merge(v, startInx, midInx, endInx);  
    }  
}
```

Mergesort

- Podemos **avaliar o desempenho** do algoritmo Mergesort analisando a sua função de recorrência:

- $$T(n) = \begin{cases} 0 & n = 1 \\ T\left(\frac{n}{2}\right) + T\left(\frac{n}{2}\right) + n & n > 1 \end{cases}$$



- Portanto, o algoritmo apresenta complexidade $\Theta(n \log(n))$.

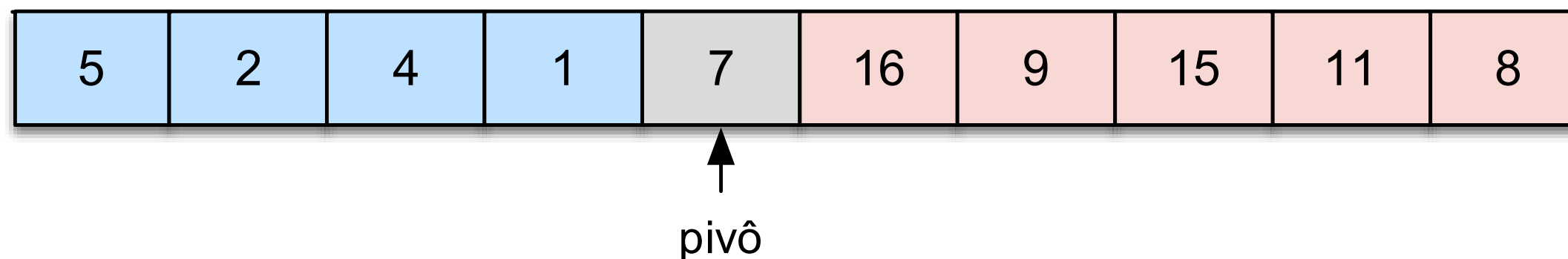
Mergesort

- Observe que o algoritmo Mergesort não requer a comparação de todos os pares
 - Mesmo no pior caso!
- Embora seja mais eficiente em tempo, Mergesort exige $O(n)$ de espaço adicional para realizar a intercalação.
- O algoritmo *insertion sort* pode apresentar um tempo menor que o Mergesort em determinadas situações:
 - A sequência for suficientemente pequena
 - Observe que o algoritmo Mergesort tende a apresentar uma constante maior (dependendo da implementação).
 - A sequência esteja próxima do estado ordenado.

Quicksort

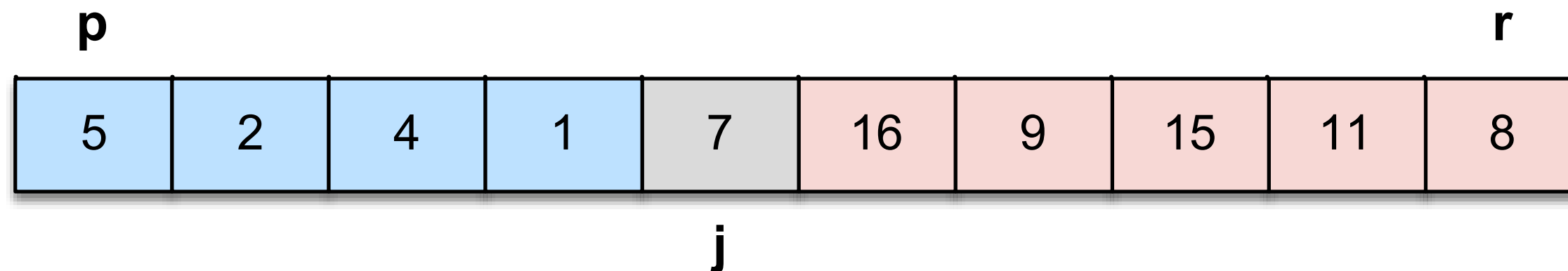
Quicksort

- O algoritmo **Quicksort** consiste em dividir a sequências em partições e ordená-las recursivamente
 - Não é requerido que as partições tenham o mesmo tamanho.
 - Cada partição precisa conter os elementos finais, ainda que fora de ordem.
- A etapa de particionamento costuma ser executada por um algoritmo auxiliar que:
 - Escolhe um elemento pivô separando a sequência em duas partições: elementos maiores e menores que o pivô.



- Portanto, o problema de particionamento pode ser definido como:
 - Dada uma sequência v e um intervalo $[p..r]$ transponha elementos desse intervalo de forma que ao retornar um índice j (pivô) tenhamos:

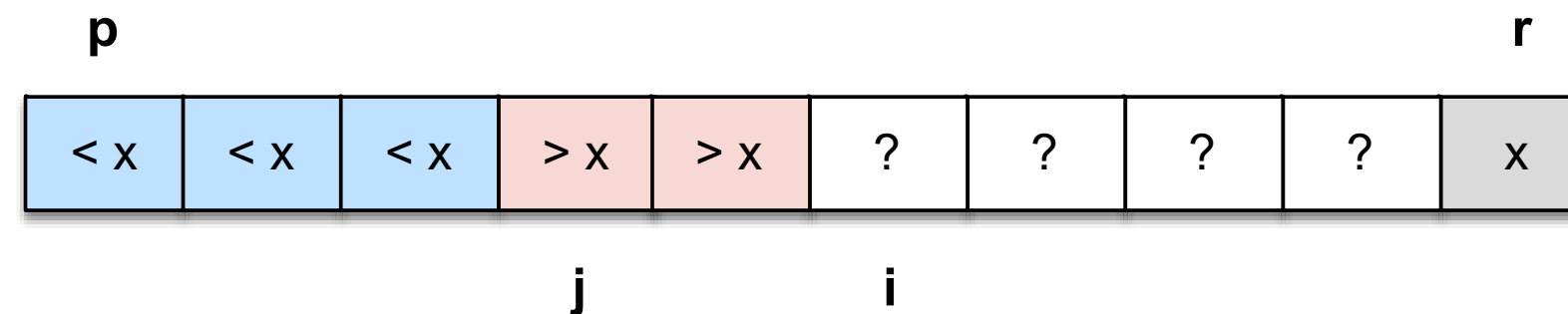
$$v[p..j-1] \leq v[j] < v[j+1..r]$$



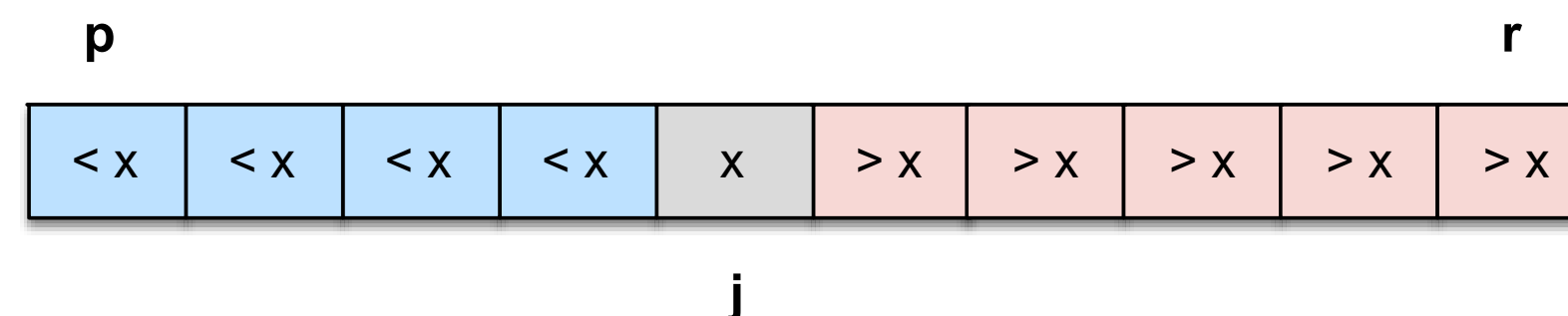
- O problema de particionamento pode ser solucionado de diversas formas
 - Buscamos algoritmos que executem a operação sem a necessidade de espaço adicional (*inplace*).

Quicksort

- Um algoritmo de particionamento comum (criado por N. Lomuto) é definido por:
 - Dada uma sequência de entrada v e um intervalo $[p..r]$.
 - Escolha o último elemento como pivô ($x = v[r]$) e inicialize $j = p$.
 - Percorra a sequência avaliando cada posição i entre p e $r - 1$:
 - Se $v[i] \leq x$ troque os valores entre i e j , e atualize o pivô $j = j + 1$.



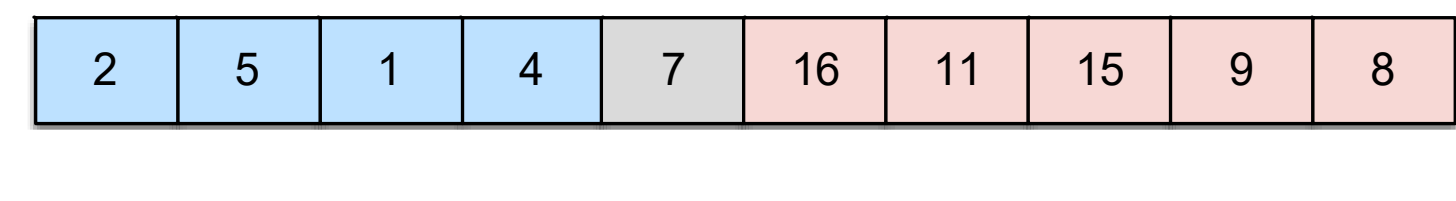
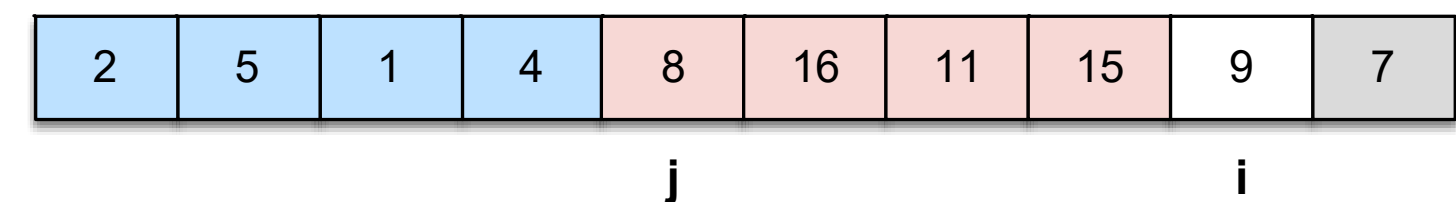
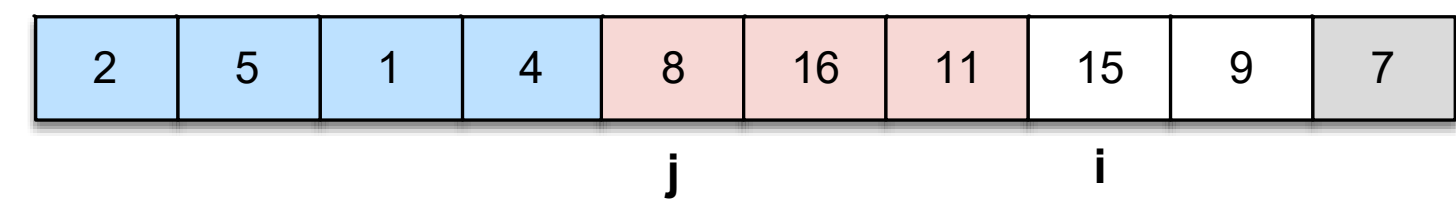
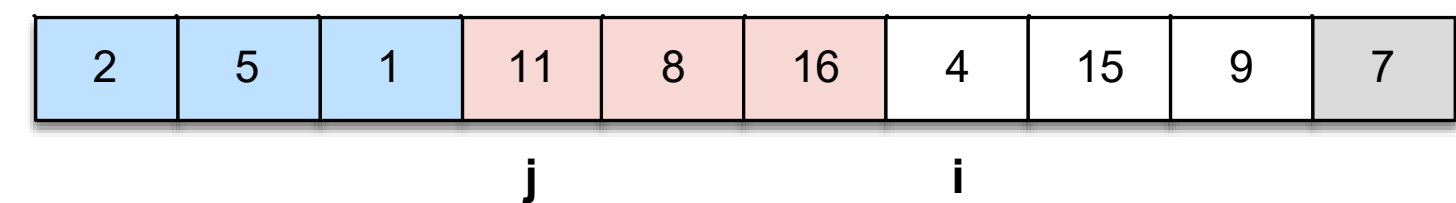
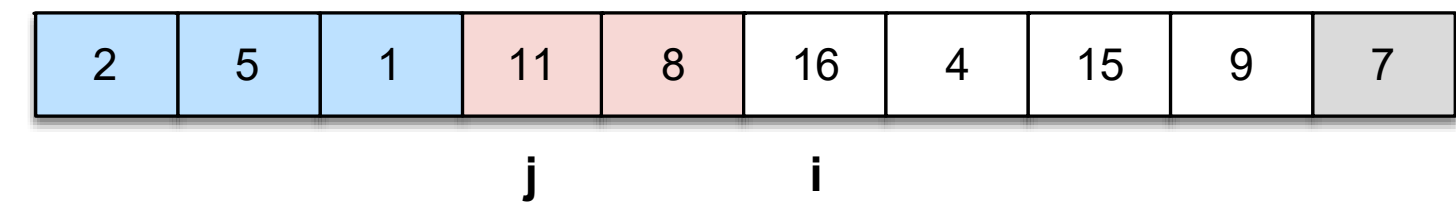
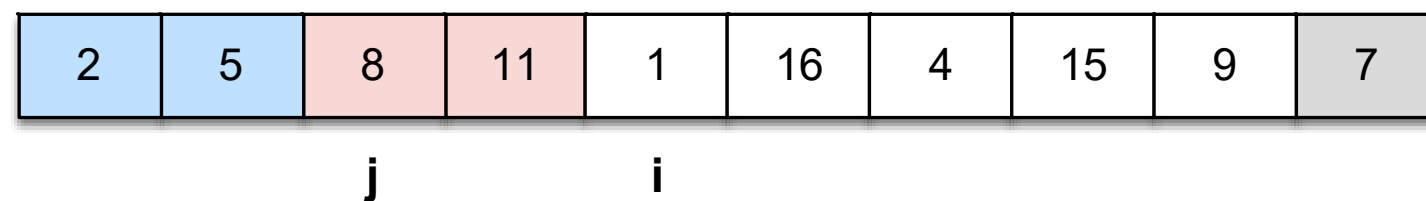
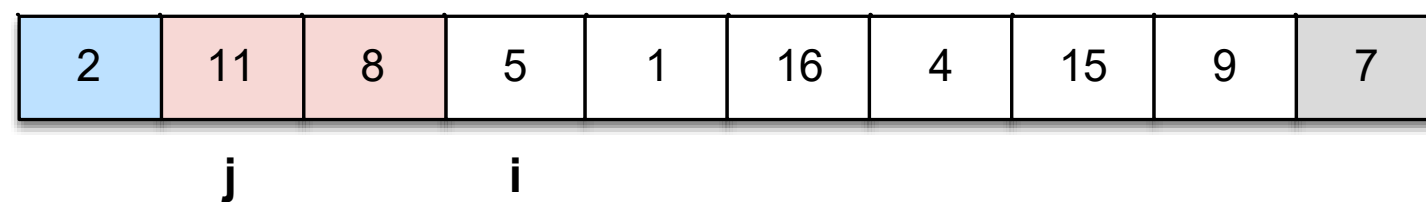
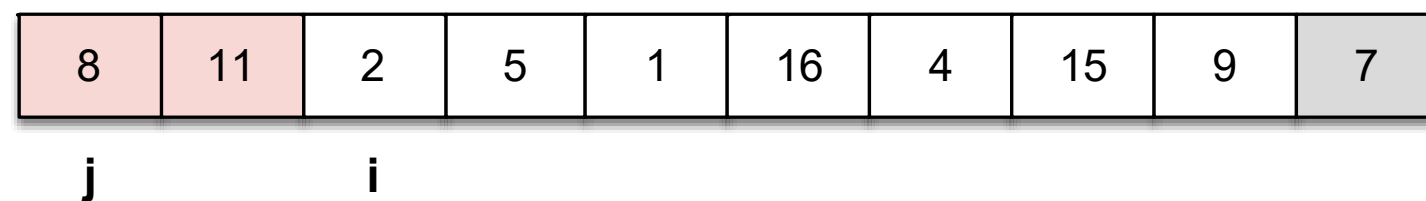
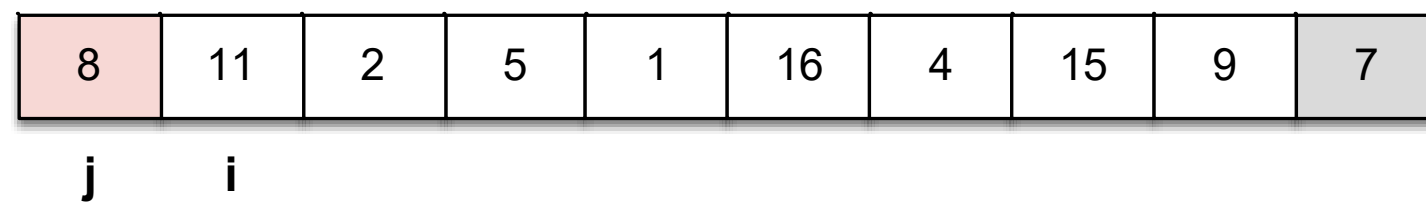
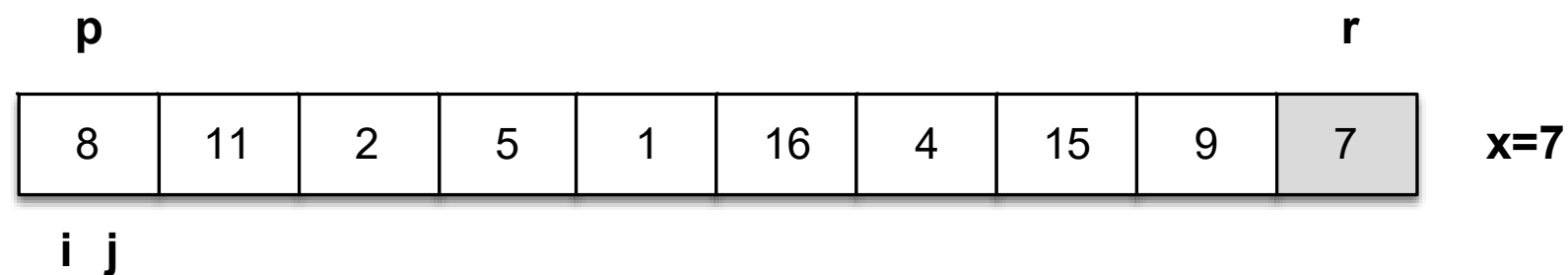
- Ao final, leve o pivô para a sua posição correta trocando os valores entre j e r .



Ordenação

Quicksort

- **Exemplo:** vamos particionar a sequência a seguir:



Quicksort

- Exemplo de implementação do algoritmo de particionamento:

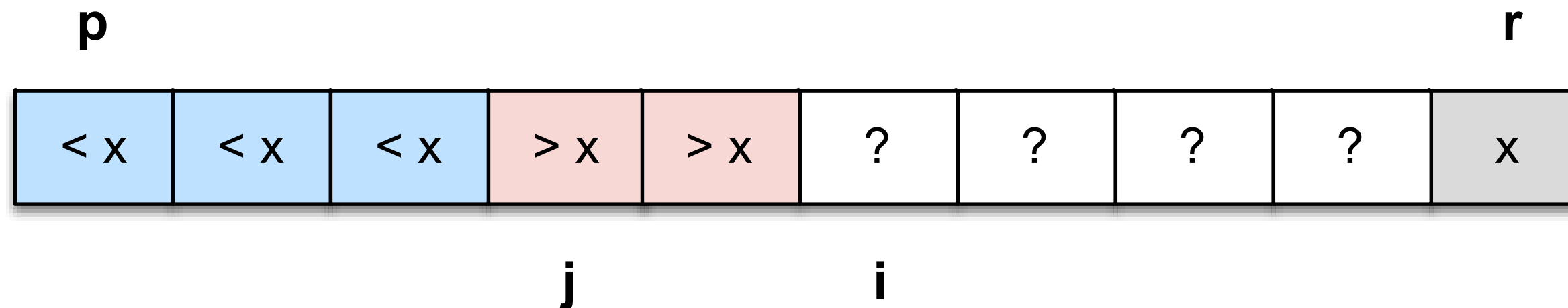
```
#define swap(v, i, j) { int temp = v[i]; v[i] = v[j]; v[j] = temp; }

int partition(int v[], int p, int r) {
    int pivot = v[r];
    int j = p;
    for (int i=p; i < r; i++) {
        if (v[i] <= pivot) {
            swap(v, i, j);
            j++;
        }
    }
    swap(v, j, r);
    return j;
}
```

- Análise do desempenho: dado que a sequência de entrada com n elementos é percorrida uma única vez executando operações com custo constante, a complexidade é $\Theta(n)$.

Quicksort

- Podemos analisar a corretude do algoritmo verificando para cada passo i as seguintes invariantes:
 - $p \leq j \leq i$
 - $v[p..j-1] \leq x$
 - $v[j..i-1] > x$
 - $v[p..r]$ é uma permutação da sequência de entrada



Quicksort

- O algoritmo **Quicksort** consiste nas seguintes etapas:

1. Particiona a sequência em:

- $v[0..j-1]$
- $v[j]$
- $v[j+1..n-1]$

2. Executa uma chamada recursiva para ordenar a sequência $v[0..j-1]$.

3. Executa uma chamada recursiva para ordenar a sequência $v[j+1..n-1]$.

8	11	2	5	1	16	4	15	9	7
2	5	1	4	7	16	11	15	9	8

2	5	1	4
---	---	---	---

16	11	15	9	8
----	----	----	---	---

2	1	4	5
---	---	---	---

8	9	15	16	11
---	---	----	----	----

2	1
---	---

5

--

9	15	16	11
---	----	----	----

1	2
---	---

--

9	11	15	16
---	----	----	----

9

15	16
----	----

15	16
----	----

1	2	4	5	7	8	9	11	15	16
---	---	---	---	---	---	---	----	----	----

Quicksort

- Exemplo de implementação:

```
void quicksort(int v[], int p, int r) {  
    if (p < r) {  
        int j = partition(v, p, r);  
        quicksort(v, p, j - 1);  
        quicksort(v, j + 1, r);  
    }  
}
```

```
quicksort(v, 0, n - 1);
```

- Podemos **avaliar o desempenho** do algoritmo Quicksort analisando a sua função de recorrência:
 - $T(n) = T(j) + T(n - j - 1) + n$
- O pior caso ocorre quando o algoritmo de particionamento retorna como pivô o maior ou menor elemento da sequência
 - Uma partição será vazia e a outra conterá $n - 1$ elementos.
$$\begin{aligned} T(n) &= T(0) + T(n - 1) + n \\ &= T(n - 2) + (n - 1) + n \\ &= T(n - 3) + (n - 2) + (n - 1) + n \\ &\dots \\ &= T(0) + 1 + 2 + 3 + \dots + (n - 1) + n \\ &= \frac{n(n+1)}{2} \end{aligned}$$
- Portanto, o algoritmo apresenta complexidade $\Theta(n^2)$ no pior caso.

- O melhor caso ocorre quando o algoritmo de particionamento retorna partições com aproximadamente o mesmo tamanho
 - $T(n) = 2T\left(\frac{n}{2}\right) + n$
- Portanto, o algoritmo apresenta complexidade $\Theta(n \log(n))$ no melhor caso.

Quicksort

- O caso médio ocorre quando o algoritmo de particionamento retorna partições com tamanhos desproporcionais (sem atingir o pior caso)
 - Considere o caso em que o algoritmo retorna partições com $0.1n$ e $0.9n$:
 - $T(n) = T\left(\frac{n}{10}\right) + T\left(\frac{9n}{10}\right) + n$
- Observe que podemos avaliar a complexidade como $\Theta(n \log(n))$
 - Qualquer divisão de proporcionalidade constante produz uma árvore com altura $\Theta(\log(n))$.
- A complexidade é a mesma do melhor caso, no entanto com uma constante maior.

- **Conclusão:** o desempenho do algoritmo Quicksort depende da escolha do pivô realizada pelo algoritmo de particionamento à cada interação.
- No pior caso a complexidade é $\Theta(n^2)$, no entanto ocorre apenas em situações extremas.
- O desempenho típico apresenta uma complexidade $\Theta(n \log(n))$.
- A constante pode ser reduzida combinando com a ordenação por inserção para sequências muito pequenas.

Quicksort

- Embora o algoritmo Mergesort apresente a complexidade $\Theta(n \log(n))$ mesmo no pior caso, a sua constante é maior que a de uma implementação tradicional do Quicksort
 - O loop interno do Quicksort pode ser implementado de forma mais eficiente.
 - A complexidade de espaço do Quicksort é $O(1)$ contra $O(n)$ do Mergesort.
- O algoritmo de partição pode ser implementado de diversas formas, minimizando a chance de ocorrer o pior caso.

Quicksort

- Um **algoritmo aleatório** tem seu comportamento definido não apenas pela entrada mas também por um gerador de números aleatórios.
- O Quicksort pode ser implementado como um algoritmo aleatório permutando a entrada ou escolhendo o pivô através de um gerador de números aleatórios
- Dessa forma aumentamos a probabilidade da partição ser razoavelmente equilibrada na média
 - Tratando cenários em que a entrada apresenta um padrão indesejado ou mesmo a ação de um usuário mal intencionado.

```
int randomPartition(int v[], int p, int r) {  
    int pivot = randomInt(p, r);  
    swap(v, pivot, r);  
    return partition(v, p, r);  
}
```

Quicksort

- A implementação básica do Quicksort pode exigir uma **pilha de execução** proporcional ao tamanho da entrada, ou seja, $\Theta(n)$.

- **Exercício 1:** em qual situação esse problema poderia ocorrer?

- O vetor de entrada estar ordenado.

- **Exercício 2:** produza um algoritmo que garanta que o tamanho da pilha seja $\Theta(\log n)$

- Compute a menor partição primeiro.
- Elimine a segunda chamada recursiva trocando o *if* por um *while*.

```
void quicksort(int v[], int p, int r) {  
    while (p < r) {  
        int j = partition(v, p, r);  
        if (j - p < r - j) {  
            quicksort(v, p, j - 1);  
            p = j + 1;  
        } else {  
            quicksort(v, j + 1, r);  
            r = j - 1;  
        }  
    }  
}
```

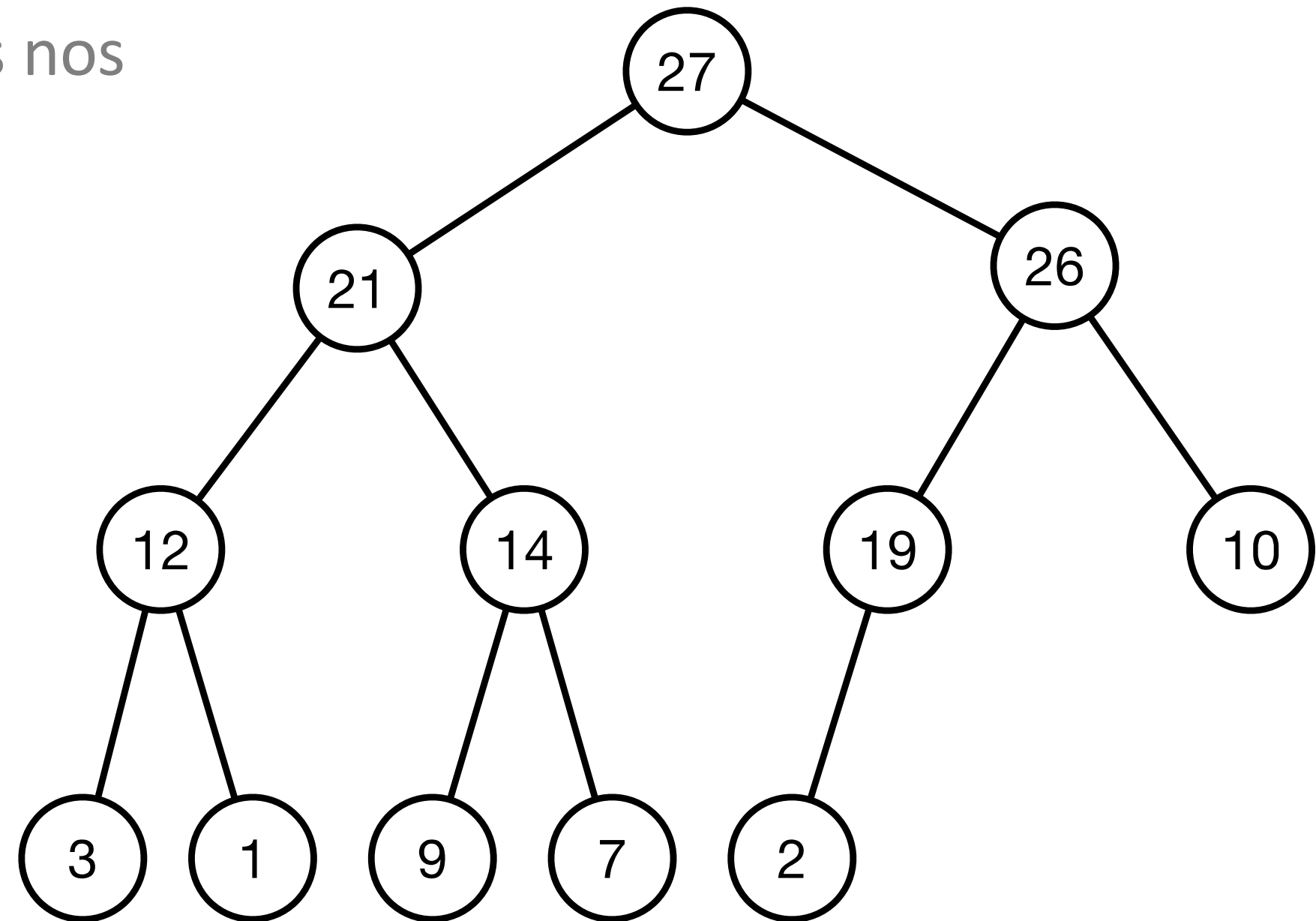
Heapsort

Heapsort

- O algoritmo **Heapsort** consiste em organizar os elementos em um heap binário e reinseri-los utilizando uma estratégia semelhante à do algoritmo de ordenação por seleção.
- O heap (monte) é uma estrutura de dados capaz de representar um vetor sob a forma de uma árvore binária, que apresenta as seguintes propriedades:
 - É uma árvore quase completa
 - Todos os níveis devem estar preenchidos exceto pelo último.
 - É mínimo ou máximo
 - Heap mínimo – cada filho será maior ou igual ao seu pai.
 - Heap máximo – cada filho será menor ou igual ao seu pai.

Heapsort

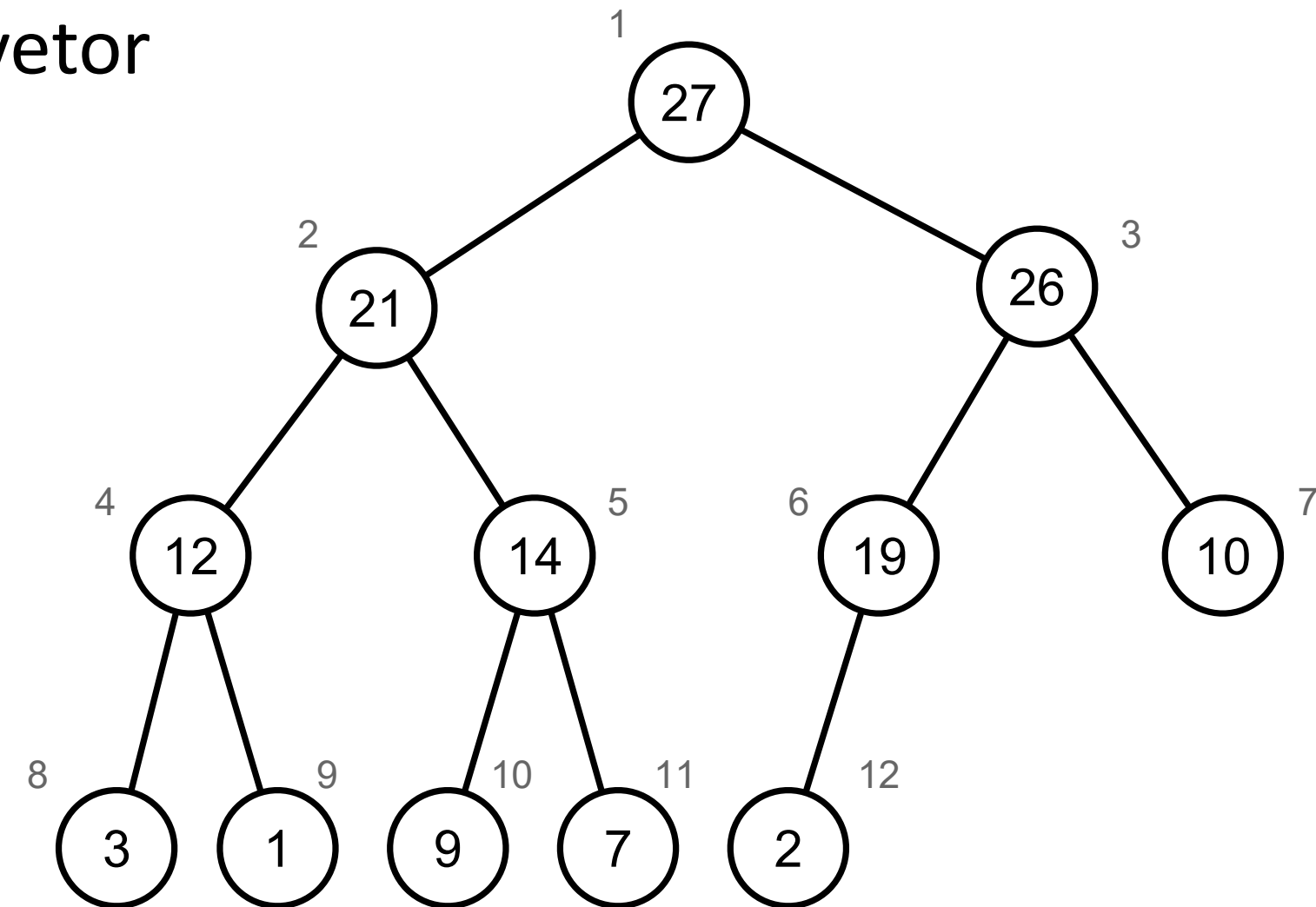
- Considere o heap ao lado:
 - Utilizaremos apenas heaps-máximos nos slides seguintes.
- A altura de um heap com n elementos é definida por $\lfloor \log_2(n) \rfloor$.
- A raiz será sempre o maior valor do heap
 - Podemos obter o maior valor armazenado em $O(1)$.



Ordenação

Heapsort

- Um heap pode ser representado por um vetor atendendo as seguintes propriedades:
 - O índice 1 é a raiz da árvore
 - A implementação continua indexando a partir de 0.
 - O pai de qualquer índice p é $p/2$
 - Exceto para a raiz, que não tem pai.
 - O filho esquerdo de p é $2p$.
 - O filho direito de p é $2p + 1$.

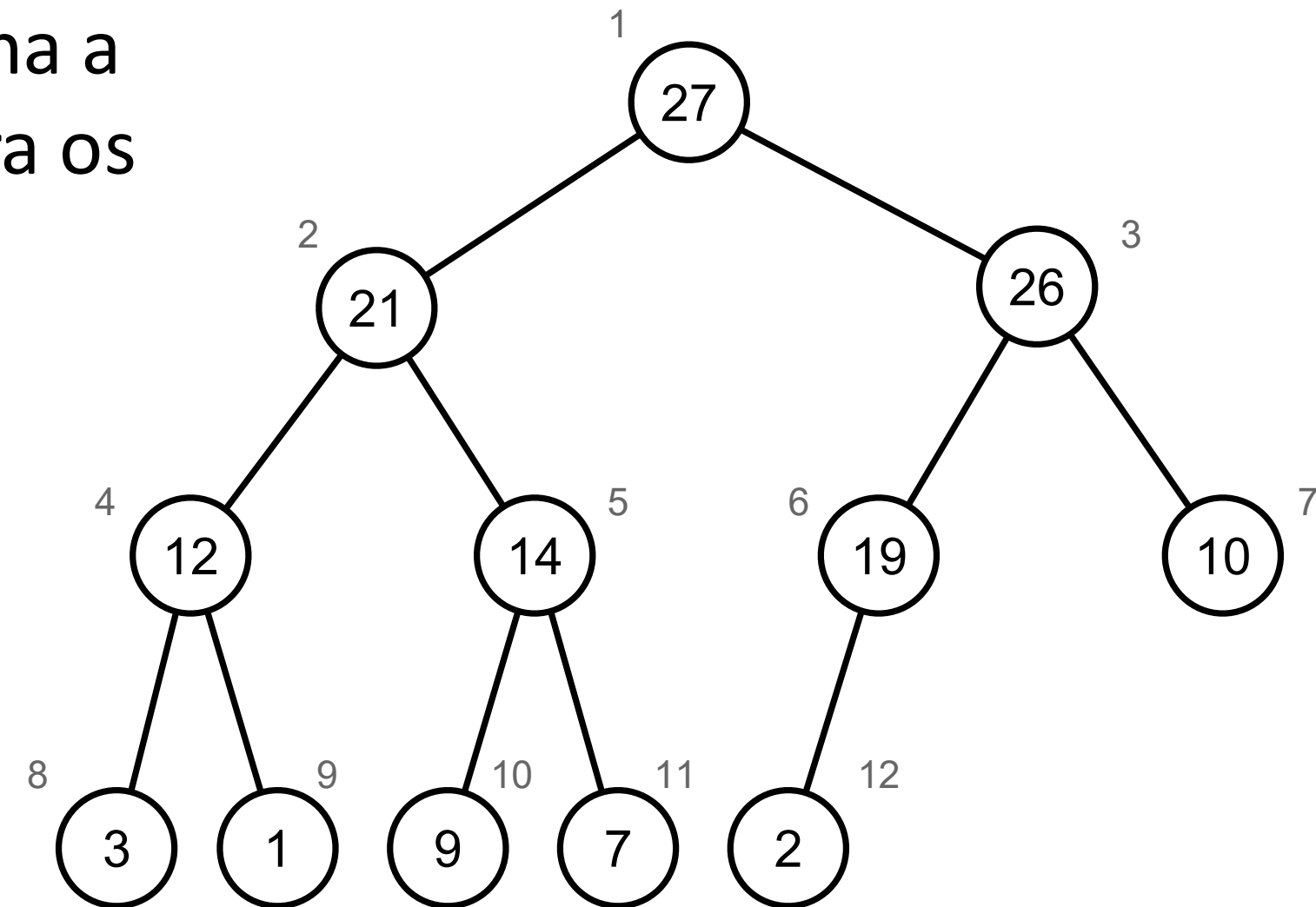


27	21	26	12	14	19	10	3	1	9	7	2
1	2	3	4	5	6	7	8	9	10	11	12

Ordenação

Heapsort

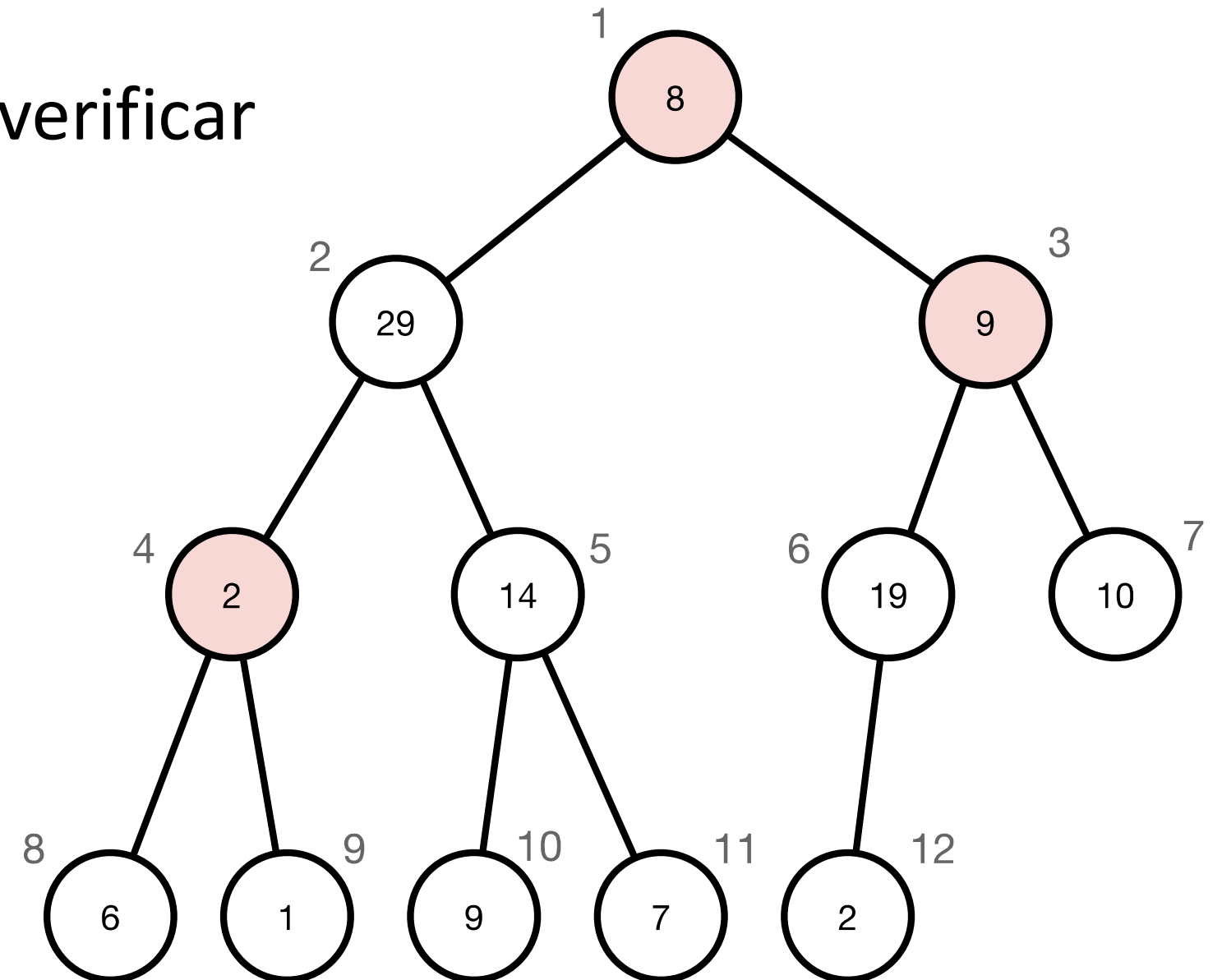
- Essa abordagem de implementação elimina a necessidade de ponteiros para o pai e para os filhos de um elemento.
- Conhecendo a primeira posição vazia do vetor a inserção pode ser realizada em $O(1)$
 - Útil para demais algoritmos que utilizam heaps.



27	21	26	12	14	19	10	3	1	9	7	2
1	2	3	4	5	6	7	8	9	10	11	12

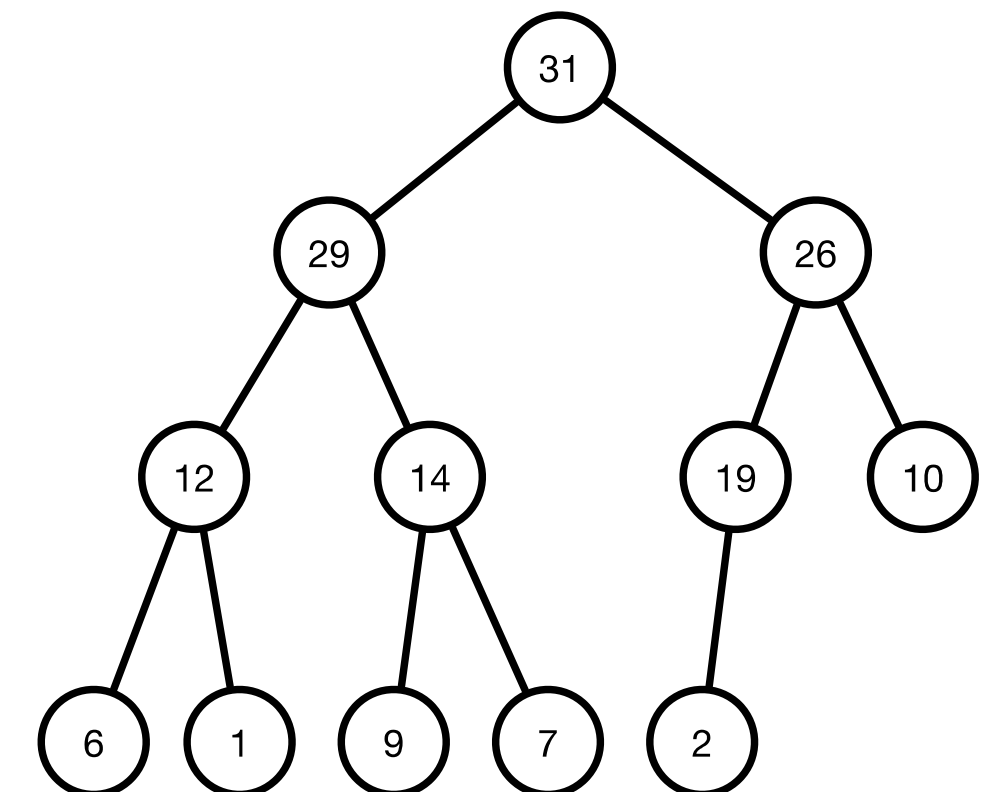
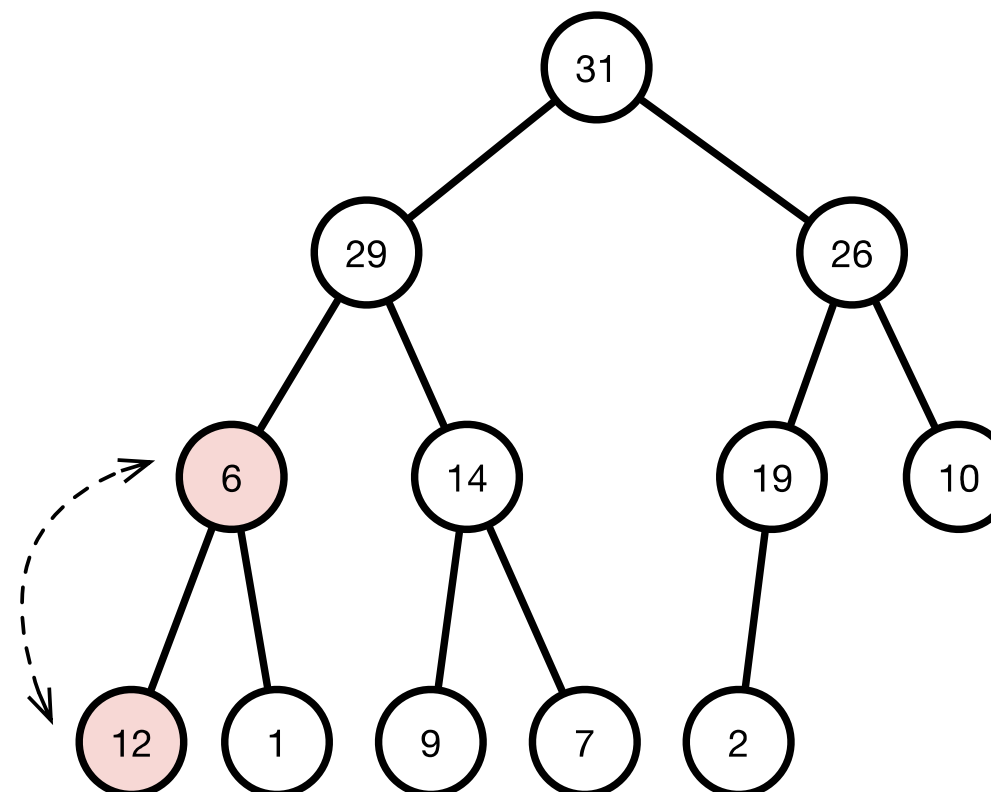
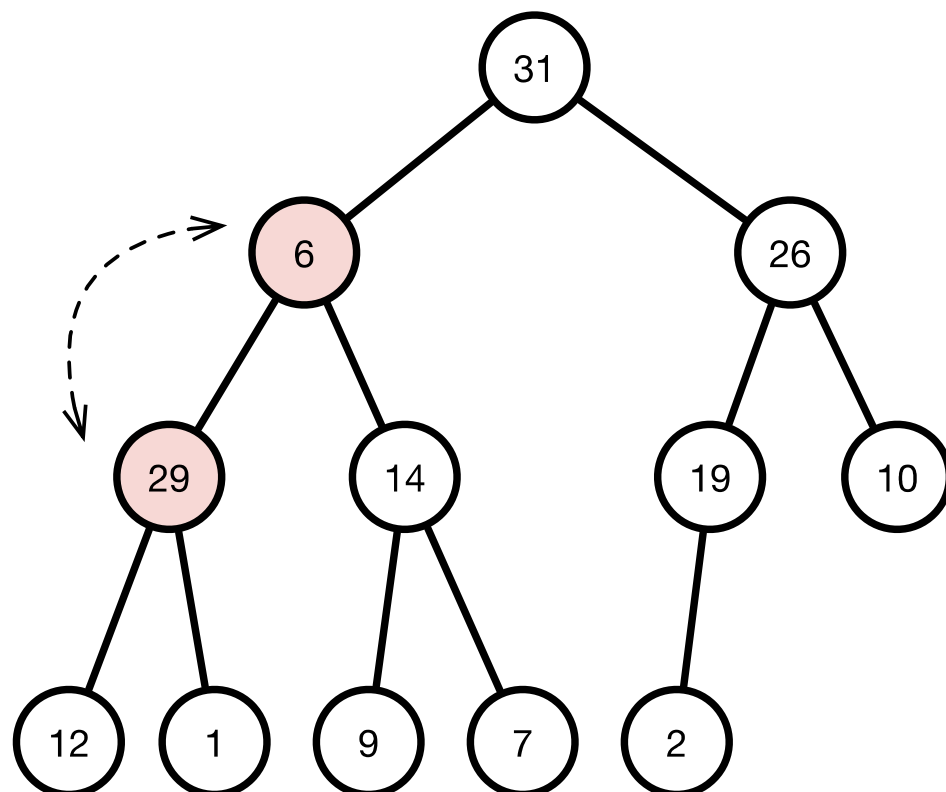
Heapsort

- Um vetor pode ser transformado em um heap permutando elementos de forma que as propriedades do heap sejam atendidas.
- O algoritmo de construção do heap consiste em verificar a propriedade $v[p] > v[p/2]$ para cada nó e corrigi-la se necessário.
- O processo inicia no último nó contendo uma folha
 - O índice desse nó pode ser obtido por $n/2$.



Heapsort

- O algoritmo **max-heapify** pode ser utilizado para realizar a correção de um determinado nó
 - Assume-se que as sub-árvores do nó são heaps-máximos.
 - Caso $v[p]$ seja menor que $v[2p]$ ou $v[2p + 1]$ escolhe o maior e executa a troca.
 - Em seguida executa max-heapify recursivamente no nó filho alterado.



Heapsort

- Exemplo de implementação:

```
void heapify(int v[], int n, int i)
{
    int inx = i;
    int leftInx = 2 * i + 1;
    int rightInx = 2 * i + 2;
    if ((leftInx < n) && (v[leftInx] > v[inx])) {
        inx = leftInx;
    }

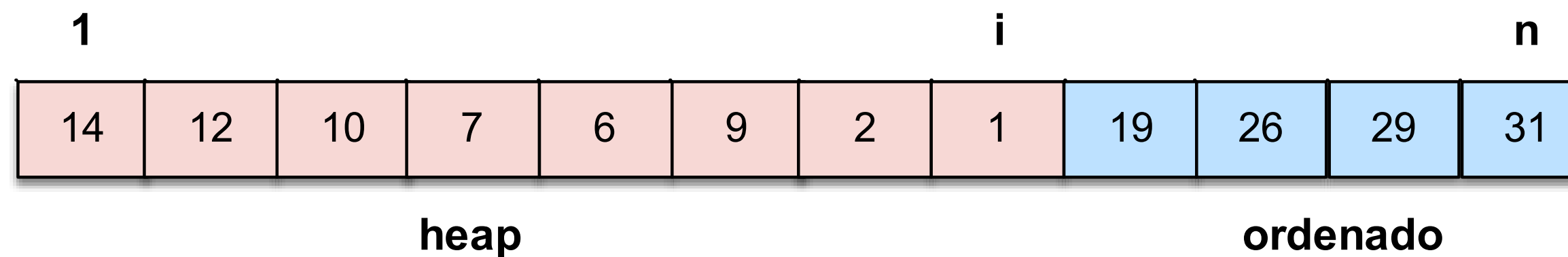
    if ((rightInx < n) && (v[rightInx] > v[inx])) {
        inx = rightInx;
    }

    if (inx != i) {
        swap(v, i, inx);
        heapify(v, n, inx);
    }
}
```

- A complexidade desse algoritmo é $O(\log(n))$.

Heapsort

- O algoritmo **Heapsort** consiste em dois passos:
 1. Organizar o vetor de entrada transformando-o em um heap.
 2. Ordenar os elementos executando os passos a seguir para cada $v[n..1]$:
 - Trocar o elemento corrente $v[i]$ pela raiz $v[1]$
 - Observe que $v[1]$ é o maior elemento do heap $v[1..i]$.
 - Corrigir o heap executando heapify para a raiz.



Heapsort

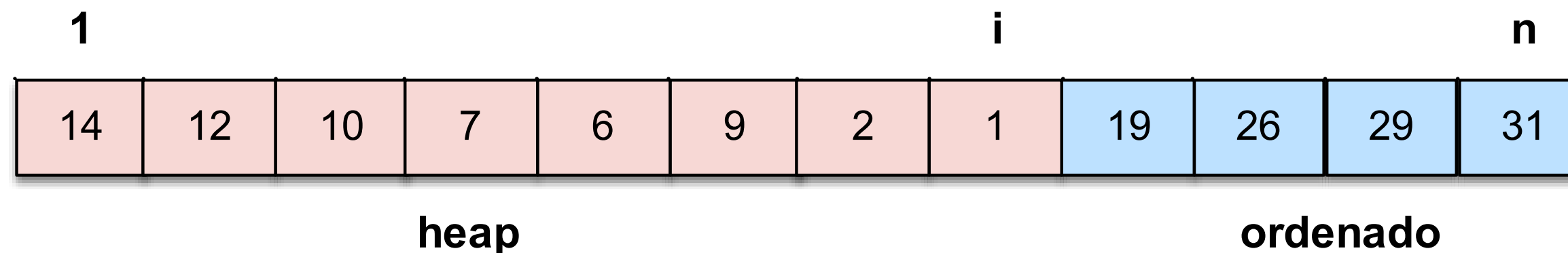
- Exemplo de implementação:

```
void buildHeap(int v[], int n) {  
    for (int i=(n/2-1); i >= 0; i--) {  
        heapify(v, n, i);  
    }  
}
```

```
void heapSort(int v[], int n) {  
    buildHeap(v, n);  
    for (int i=n-1; i > 0; i--) {  
        swap(v, 0, i);  
        heapify(v, i, 0);  
    }  
}
```

Heapsort

- Podemos analisar a corretude do algoritmo verificando para cada passo i as seguintes invariantes:
 - O vetor $v[1..i]$ é um heap.
 - $v[1..i] < v[i + 1..n]$.
 - $v[i + 1..n]$ está em ordem crescente.
 - $v[1..n]$ é uma permutação da sequência de entrada.

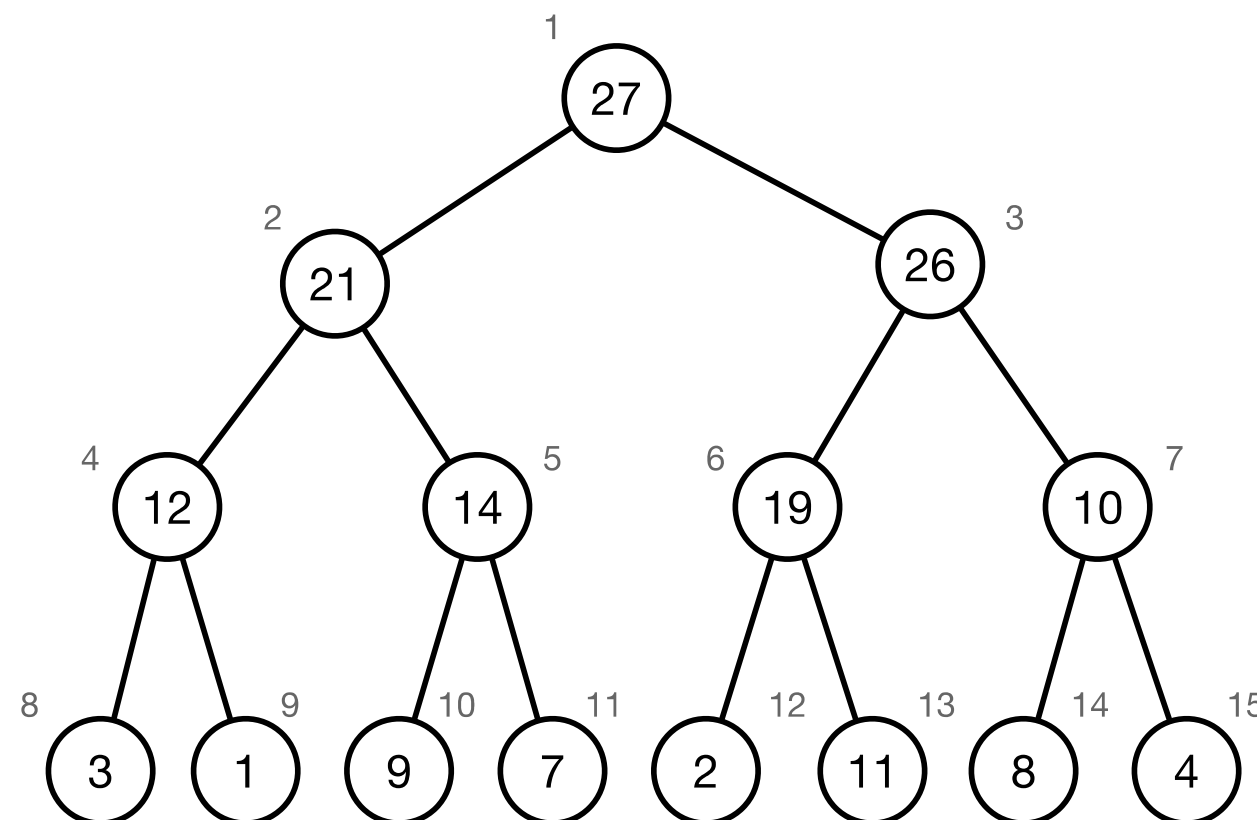


Heapsort

- Podemos **avaliar o desempenho** do algoritmo Heapsort analisando cada uma das suas etapas individualmente:
 - Construção do heap:
 - Executa heapify em um vetor de $\sim n/2$ posições, logo $O(n \log(n))$
 - Ordenação:
 - Executa heapify para cada elemento em um vetor de $n - 1$ posições, logo $O(n \log(n))$
- Portanto, a complexidade do algoritmo é $\Theta(n \log(n))$.

Heapsort

- Como podemos melhorar a análise da etapa de construção do heap?
 - Vamos calcular o número de operações aproximadamente:
 - Cada nível i de baixo para cima tem $\sim \frac{n}{2^i}$ nós.
 - Portanto o custo total é $\sum_{i=1}^{\log(n)} i \cdot \frac{n}{2^i} = O(n)$



nível	# nós
4	$\sim \frac{n}{2^4}$
3	$\sim \frac{n}{2^3}$
2	$\sim \frac{n}{2^2}$
1	$\sim \frac{n}{2^1}$

Ordenação em tempo linear

- Até o momento vimos algoritmos eficientes de ordenação capazes de ordenar uma sequência de n elementos em $\Theta(n \log(n))$.
- Mesmo no melhor caso a complexidade desses algoritmos é $\Omega(n \log(n))$.
- Todos eles compartilham uma característica interessante: a ordenação é definida pela comparação entre os elementos da sequência.
- Vamos agora conhecer algoritmos que utilizam outras abordagens (ex: baseada em contagem), e conseguem apresentar uma complexidade menor que $\Theta(n \log(n))$
 - Naturalmente, esses algoritmos apresentam outras limitações.

Counting sort

- O algoritmo de **ordenação por contagem** (*counting sort*) consiste em computar para cada elemento quantos elementos menores existem na sequência
 - Sabendo que o elemento v_i possui j elementos menores do que ele, podemos definir sua posição final como $j + 1$.
- Definição do problema: desejamos ordenar os elementos do vetor $v[0..n - 1]$ considerando as seguintes restrições:
 - Os elementos são números inteiros.
 - Os números estão presentes no intervalo $[0..k - 1]$
 - O universo possui tamanho k .
 - k é pequeno.

- Dada uma sequência de entrada:

v	5	2	4	5	1	3	1	1	5	0	4
-----	---	---	---	---	---	---	---	---	---	---	---

$n = 11$
 $k = 6$

- Podemos calcular a frequência de cada elemento:

	0	1	2	3	4	5
f	1	3	1	1	2	3

- Essa sequência auxiliar possui tamanho k .

Counting sort

- Com base na sequência de frequências f podemos facilmente gerar uma sequência sf contendo em sf_i o número de elementos menores do que o primeiro elemento com valor i
 - Observe que: $sf_i = \sum_{j=0}^{i-1} f[j] = f[i-1] + sf[i-1]$, para $i > 0$.

	0	1	2	3	4	5	6
f	1	3	1	1	2	3	
sf	0	1	4	5	6	8	11

- Com base na sequência sf podemos gerar o vetor $v[0..n-1]$ ordenado:

0	1	2	3	4	5	6	7	8	9	10
0	1	1	1	2	3	4	4	5	5	5

Counting sort

■ Exemplo de implementação:

```
void countingSort(int v[], int n, int k) {
    int fs[k + 1];
    int temp[n];
    for (int j = 0; j <= k; j++) {
        fs[j] = 0;
    }
    for (int i = 0; i < n; i++) {
        fs[v[i] + 1] += 1;
    }
    for (int j = 1; j <= k; j++) {
        fs[j] += fs[j - 1];
    }
    for (int i = 0; i < n; i++) {
        int j = v[i];
        temp[fs[j]] = v[i];
        fs[j]++;
    }
    for (int i = 0; i < n; i++) {
        v[i] = temp[i];
    }
}
```

- Podemos **avaliar o desempenho** do algoritmo *Counting Sort* através da seguinte função:

$$\begin{aligned} f(n, k) &= c_1 k + c_2 n + c_3 k + c_4 n + c_5 n \\ &= (c_1 + c_3)k + (c_2 + c_4 + c_5)n \\ &= \Theta(k + n) \end{aligned}$$

- Exige $O(n + k)$ de espaço adicional.
- Portanto, se k for muito pequeno a complexidade será $\Theta(n)$.
- É considerado um algoritmo eficiente para ordenar sequências com elementos repetidos.

Radix sort

Radix sort

- O algoritmo **Radix sort** consiste em ordenar os elementos de uma sequência digito à digito, do menos significativo para o mais significativo
 - A solução é baseada no algoritmo de ordenação por contagem.
- Considera que cada elemento é uma sequência com w dígitos
 - Todos os elementos possuem o mesmo número de dígitos.
- Embora o termo dígito remeta à números do conjunto $[0..9]$, podemos utilizar qualquer valor que possa ser mapeado em um inteiro
 - O exemplo a seguir irá utilizar caracteres ASCII.
 - O tamanho do universo continua sendo definido como k .

Radix sort

- Vamos ordenar um conjunto de placas de carro como exemplo:

$$n = 10$$

$$k = 36 *$$

$$w = 7$$

L A Y 5 0 2 1	D E M 2 2 0 1	L A Y 5 0 2 1	L I T 1 2 3 4	C A L 8 8 5 4	C A L 8 8 5 4	B A N 9 3 2 3
D E M 2 2 0 1	B A Y 7 2 1 8	F I N 5 0 2 8	D E M 2 2 0 1	D E M 2 2 0 1	B A N 9 3 2 3	B A Y 7 2 1 8
B A N 9 3 2 3	L A Y 5 0 2 1	D E M 2 2 0 1	B D T 2 3 2 9	F I N 5 0 2 8	L A Y 5 0 2 1	B D T 2 3 2 9
O D S 9 9 8 3	B A N 9 3 2 3	B A Y 7 2 1 8	O E S 3 9 2 6	B A N 9 3 2 3	B A Y 7 2 1 8	C A L 8 8 5 4
C A L 8 8 5 4	O E S 3 9 2 6	L I T 1 2 3 4	L A Y 5 0 2 1	O E S 3 9 2 6	O D S 9 9 8 3	D E M 2 2 0 1
L I T 1 2 3 4	F I N 5 0 2 8	B A N 9 3 2 3	F I N 5 0 2 8	O D S 9 9 8 3	B D T 2 3 2 9	F I N 5 0 2 8
O E S 3 9 2 6	B D T 2 3 2 9	B D T 2 3 2 9	B A Y 7 2 1 8	L I T 1 2 3 4	D E M 2 2 0 1	L A Y 5 0 2 1
F I N 5 0 2 8	L I T 1 2 3 4	C A L 8 8 5 4	C A L 8 8 5 4	B D T 2 3 2 9	O E S 3 9 2 6	L I T 1 2 3 4
B A Y 7 2 1 8	C A L 8 8 5 4	O E S 3 9 2 6	B A N 9 3 2 3	L A Y 5 0 2 1	F I N 5 0 2 8	O D S 9 9 8 3
B D T 2 3 2 9	O D S 9 9 8 3	O D S 9 9 8 3	O D S 9 9 8 3	B A Y 7 2 1 8	L I T 1 2 3 4	O E S 3 9 2 6

- Exemplo de implementação:

```
void radixSort(unsigned char * v[], int n, int W, int K) {  
    int fp [K + 1];  
    unsigned char* aux[n];  
    for (int w = W - 1; w >= 0; w--) {  
        for (int j = 0; j <= K; j++) { fp[j] = 0; }  
        for (int i = 0; i < n; i++) {  
            fp[v[i][w] + 1] += 1;  
        }  
        for (int j = 1; j <= K; j++) { fp[j] += fp[j - 1]; }  
        for (int i = 0; i < n; i++) {  
            int j = v[i][w];  
            aux[fp[j]] = v[i];  
            fp[j]++;  
        }  
        for (int i = 0; i < n; i++) { v[i] = aux[i]; }  
    }  
}
```

- Podemos **avaliar o desempenho** do algoritmo Radix Sort através da seguinte função:

$$\begin{aligned} f(n, k, w) &= w(c_1k + c_2n + c_3k + c_4n + c_5n) \\ &= w((c_1 + c_3)k + (c_2 + c_4 + c_5)n) \\ &= \Theta(w(k + n)) \end{aligned}$$

- Exige $O(n + k)$ de espaço adicional.
- Se k e w forem pequenos a complexidade pode ser avaliada como $\Theta(n)$.

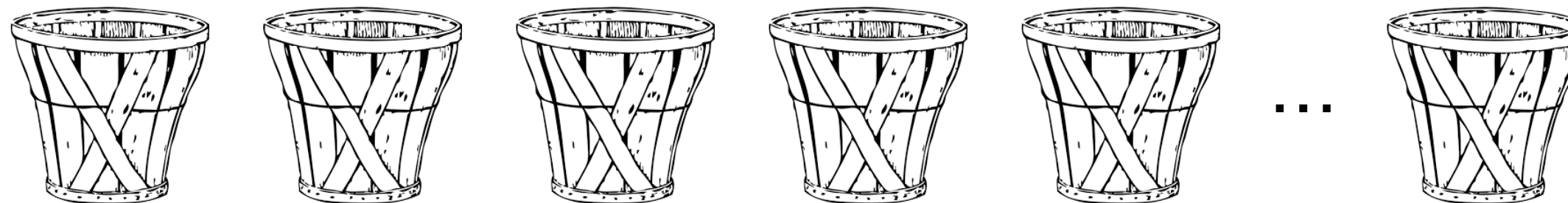
Bucket sort

Bucket sort

- O algoritmo de **ordenação por balde** (*bucket sort*) consiste em:
 - Dividir uma sequência v com n elementos - números no intervalo $[0,1)$

0	1	2	3	4	5	6	7	8	9
0.75	0.12	0.91	0.31	0.80	0.20	0.37	0.73	0.54	0.32

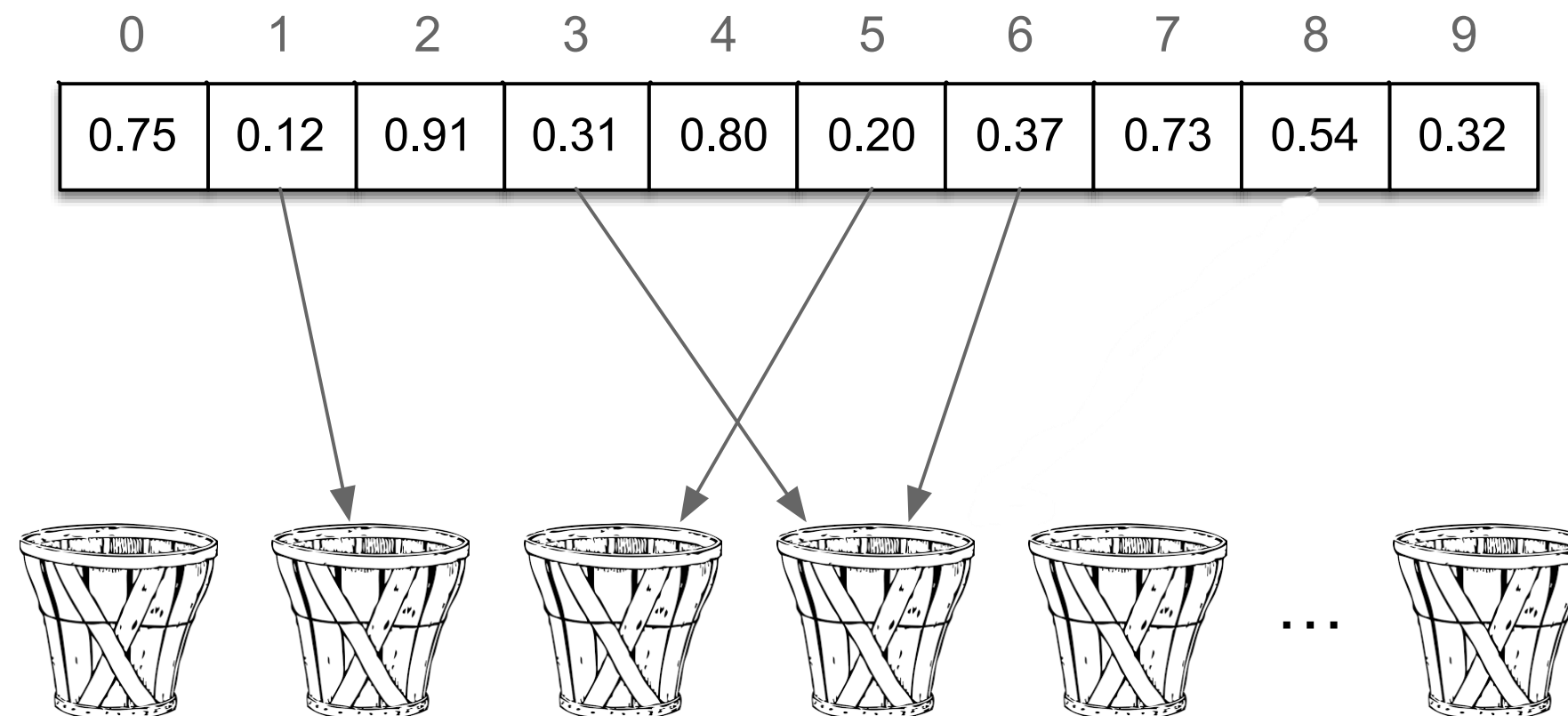
- Em n grupos (baldes)



- Pressupõe-se que os valores são distribuídos uniformemente no intervalo $[0,1)$.

Bucket sort

- Considerando um vetor de baldes b com tamanho n , o destino de cada elemento i é definido por $b[\lfloor n \cdot v[i] \rfloor]$



- Após a distribuição em baldes ordenamos cada balde e percorremos cada um gerando a sequência ordenada.

Bucket sort

- Ideia geral:
 - Criar um vetor b com tamanho n
 - Cada elemento de b é uma lista encadeada
 - Para cada elemento $v[i]$
 - Inserir em b na posição $\lfloor n \cdot v[i] \rfloor$
 - Para cada lista $b[i]$
 - Ordenar os seus elementos utilizando algum algoritmo conhecido
 - Ex: Insertion Sort
 - Para cada lista $b[i]$
 - Para cada elemento j de $b[i]$
 - Inserir na lista original v

- Exemplo de implementação:

```
void bucketSort(float v[], int n) {  
    vector<float> b[n];  
    for (int i = 0; i < n; i++) {  
        int inx = n * v[i];  
        b[inx].push_back(v[i]);  
    }  
    for (int i = 0; i < n; i++) {  
        insertionSort(b[i]);  
    }  
    int index = 0;  
    for (int i = 0; i < n; i++) {  
        for (int j = 0; j < b[i].size(); j++) {  
            v[index++] = b[i][j];  
        }  
    }  
}
```

- Podemos **avaliar o desempenho** do algoritmo através da seguinte função:

$$T(n) = \Theta(n) + \sum_{i=0}^{n-1} n_i^2$$

- Portanto, temos que:
 - O melhor caso é $\Theta(n)$ - cada balde recebe exatamente um elemento.
 - O pior caso é $\Theta(n^2)$ - um único balde recebe n elementos.
 - O caso médio é $\Theta(n)$ - considerando a distribuição uniforme esperada, poucos elementos caem no mesmo balde.
- Exige $O(n)$ de espaço adicional.

- **Exercício 1:** crie um algoritmo capaz de intercalar os elementos de m sequências de forma ordenada. As sequências de entrada já estão ordenadas, e o algoritmo deverá apresentar uma complexidade $O(n \log(m))$, sendo n o número total de elementos.
- **Ideia:** construir um heap mínimo contendo exatamente uma entrada por sequência não vazia; cada entrada é uma tripla (valor, índice da sequência de origem, posição do valor dentro dela). Inicializá-lo com o primeiro elemento de cada sequência. Repetir até o heap esvaziar: (1) remover a raiz e acrescentar seu valor ao final da saída; (2) se a sequência de origem da raiz ainda tiver elemento na posição seguinte, inserir esse elemento no heap e corrigí-lo. A invariante que garante a corretude: a cada instante, o heap contém o menor elemento ainda não consumido de cada sequência; como cada sequência está ordenada, o menor desses m candidatos é o menor elemento global restante, e aplicar a correção do heap custa $O(\log(m))$.

- **Exercício 1:** crie um algoritmo capaz de intercalar os elementos de m sequências de forma ordenada. As sequências de entrada já estão ordenadas, e o algoritmo deverá apresentar uma complexidade $O(n \log(m))$, sendo n o número total de elementos.

```
mergeMSeq(seqs[m]):  
    H = heap_create()           // entradas (valor, s, inx)  
    for s = 0 to m-1:  
        if tamanho(seqs[s]) > 0:  
            heap_insert(H, (seqs[s][0], s, 0))  
    r = []  
    while H not empty:  
        (val, s, inx) = heap_remove_min(H)  
        r.append(val)  
        if inx + 1 < size(seqs[s]):  
            heap_insert(H, (seqs[s][inx+1], s, inx+1))  
    return r
```

- **Exercício 1:** crie um algoritmo capaz de intercalar os elementos de m sequências de forma ordenada. As sequências de entrada já estão ordenadas, e o algoritmo deverá apresentar uma complexidade $O(n \log m)$, sendo n o número total de elementos.
- **Análise:** o heap nunca excede m entradas, então *heap_insert* e *heap_remove* custam $O(\log m)$. Cada um dos n elementos é inserido exatamente uma vez e removido exatamente uma vez: $2n$ operações de heap, total $O(n \log m)$. Espaço adicional: $O(m)$ para o heap.

- **Exercício 2:** uma plataforma de e-commerce armazena n avaliações de produtos, cada uma com nota inteira de 0 a 100. O time de análise quer um painel que responda instantaneamente a perguntas como "*quantas avaliações têm nota entre 60 e 80?*", para qualquer intervalo $[a..b]$. Projete uma estrutura que, após pré-processamento de custo $O(n)$, responda cada consulta em $O(1)$.
- **Ideia:** o universo de notas é pequeno e fixo ($k = 101$), o que possibilita uma abordagem baseada em contagem. Faremos um pré-processamento em duas etapas: (1) construir o vetor de frequências $f[0..100]$, em que $f[j]$ = número de avaliações com nota exatamente j , percorrendo a entrada uma única vez; e (2) construir o vetor acumulado $sf[0..100]$, em que $sf[j] = f[0] + \dots + f[j]$ = número de avaliações com nota $\leq j$. A consulta $[a..b]$ é respondida por subtração: (avaliações $\leq b$) menos (avaliações $\leq a - 1$), ou seja, $sf[b] - sf[a - 1]$; o caso $a = 0$ é tratado à parte, retornando $sf[b]$ diretamente.

- **Exercício 2:** uma plataforma de e-commerce armazena n avaliações de produtos, cada uma com nota inteira de 0 a 100. O time de análise quer um painel que responda instantaneamente a perguntas como "*quantas avaliações têm nota entre 60 e 80?*", para qualquer intervalo $[a..b]$. Projete uma estrutura que, após pré-processamento de custo $O(n)$, responda cada consulta em $O(1)$.

```
prepare(v[], n):  
    for j = 0 to 100: f[j] = 0  
    for i = 0 to n-1: f[v[i]] += 1  
    sf[0] = f[0]  
    for j = 1 to 100:  
        sf[j] = sf[j-1] + f[j]
```

```
compute(a, b):  
    if a == 0: return sf[b]  
    return sf[b] - sf[a-1]
```

- **Exercício 2:** uma plataforma de e-commerce armazena n avaliações de produtos, cada uma com nota inteira de 0 a 100. O time de análise quer um painel que responda instantaneamente a perguntas como "*quantas avaliações têm nota entre 60 e 80?*", para qualquer intervalo $[a..b]$. Projete uma estrutura que, após pré-processamento de custo $O(n)$, responda cada consulta em $O(1)$.
- **Análise:** o pré-processamento custa $\Theta(n + k)$, dado que consistem em uma passada sobre as n avaliações, mais duas varreduras sobre os $k = 101$ contadores; e, como k é constante, temos $\Theta(n)$. Cada consulta realiza no máximo uma subtração: $O(1)$. Espaço adicional: $O(k) = O(1)$.

- **Exercício 3:** um sistema antifraude recebe um vetor com n transações, cada uma com um valor em reais. A auditoria definiu dois tetos $x_1 < x_2$: transações com valor até x_1 são aprovadas automaticamente, entre x_1 e x_2 exigem revisão manual, e acima de x_2 são bloqueadas. O resultado deve ser entregue no próprio vetor de entrada, com espaço adicional $O(1)$: as aprovadas no início, as de revisão no meio e as bloqueadas no final. Não é exigida ordem dentro de cada região. Projete um algoritmo com custo $\Theta(n)$ que realize uma única varredura do vetor.
- **Ideia:** generalizar o particionamento do quicksort usando três índices: lo para fronteira das aprovadas, mid para a próxima posição a classificar, e hi para a fronteira dos bloqueados. Considere a invariante: $v[0..lo-1] \leq x_1$; $x_1 < v[lo..mid-1] \leq x_2$; $v[mid..hi]$ ainda não classificado; $v[hi+1..n-1] > x_2$. Enquanto $mid \leq hi$: se $v[mid] \leq x_1$, trocar com $v[lo]$ e avançar lo e mid ; se $v[mid] > x_2$, trocar com $v[hi]$ e recuar hi ; caso contrário, apenas avançar mid .

- **Exercício 3:** um sistema antifraude recebe um vetor com n transações, cada uma com um valor em reais. A auditoria definiu dois tetos $x_1 < x_2$: transações com valor até x_1 são aprovadas automaticamente, entre x_1 e x_2 exigem revisão manual, e acima de x_2 são bloqueadas. O resultado deve ser entregue no próprio vetor de entrada, com espaço adicional $O(1)$: as aprovadas no início, as de revisão no meio e as bloqueadas no final. Não é exigida ordem dentro de cada região. Projete um algoritmo com custo $\Theta(n)$ que realize uma única varredura do vetor.

```
trriage_transactions(v[], n, x1, x2):
    lo = 0
    mid = 0
    hi = n - 1
    while mid <= hi:
        if v[mid] <= x1:
            swap(v, mid, lo)
            lo += 1
            mid += 1
        else if v[mid] > x2:
            swap(v, mid, hi)
            hi -= 1
        else:
            mid += 1
    return (lo, hi)
```

- **Exercício 3:** um sistema antifraude recebe um vetor com n transações, cada uma com um valor em reais. A auditoria definiu dois tetos $x_1 < x_2$: transações com valor até x_1 são aprovadas automaticamente, entre x_1 e x_2 exigem revisão manual, e acima de x_2 são bloqueadas. O resultado deve ser entregue no próprio vetor de entrada, com espaço adicional $O(1)$: as aprovadas no início, as de revisão no meio e as bloqueadas no final. Não é exigida ordem dentro de cada região. Projete um algoritmo com custo $\Theta(n)$ que realize uma única varredura do vetor.
- **Análise:** a cada iteração, ou mid avança ou hi recua, logo a distância $hi - mid$ decresce estritamente e o laço executa no máximo n iterações com custo constante, portanto $\Theta(n)$ no total. Espaço adicional: três índices e a variável temporária do swap, $O(1)$.

- **Exercício 4:** um sistema distribuído coleta eventos de vários servidores em um arquivo central. Por atrasos de rede, os eventos chegam quase em ordem cronológica: cada evento está a no máximo d posições do seu lugar correto ($d \ll n$). Projete um algoritmo com custo $O(n \log d)$ e espaço adicional $O(d)$ que produza o arquivo ordenado. Também responda: por que não usar direto o insertion sort, que é eficiente para sequências quase ordenadas?
- **Ideia:** usar um heap mínimo como janela deslizante de tamanho $d + 1$, inicializado com os primeiros $d + 1$ eventos da lista. Observe que o elemento que pertence à posição p da saída está na entrada em um índice $\leq p + d$, logo já foi lido quando a janela alcança essa posição. Em seguida deve-se repetir: (1) remover a raiz do heap e gravá-la na próxima posição livre da saída; (2) inser o próximo evento lido da lista no heap. Ao esgotar a entrada, esvaziar o heap removendo a raiz repetidamente.

- **Exercício 4:** um sistema distribuído coleta eventos de vários servidores em um arquivo central. Por atrasos de rede, os eventos chegam quase em ordem cronológica: cada evento está a no máximo d posições do seu lugar correto ($d \ll n$). Projete um algoritmo com custo $O(n \log d)$ e espaço adicional $O(d)$ que produza o arquivo ordenado. Também responda: por que não usar direto o insertion sort, que é eficiente para sequências quase ordenadas?

```
sort_almost_sorted(v[], n, d):  
    H = heap_create()  
    for i = 0 to d:  
        heap_insert(H, v[i])  
    out = 0  
    for i = d+1 to n-1:  
        v[out] = heap_remove_min(H)  
        out += 1  
        heap_insert(H, v[i])  
    while H not empty:  
        v[out] = heap_remove_min(H)  
        out += 1
```

- **Exercício 4:** um sistema distribuído coleta eventos de vários servidores em um arquivo central. Por atrasos de rede, os eventos chegam quase em ordem cronológica: cada evento está a no máximo d posições do seu lugar correto ($d \ll n$). Projete um algoritmo com custo $O(n \log d)$ e espaço adicional $O(d)$ que produza o arquivo ordenado. Também responda: por que não usar direto o insertion sort, que é eficiente para sequências quase ordenadas?
- **Análise:** o heap nunca excede $d + 1$ elementos, logo cada operação custa $O(\log d)$. São n remoções e n inserções, logo $O(n \log d)$. Espaço adicional: $O(d)$ para o heap. Sobre o insertion sort: nesse cenário cada elemento desloca até d posições, custo $O(n \cdot d)$; é competitivo para d minúsculo, mas $O(n \log d)$ cresce mais lentamente em d . Por fim, a saída pode ser o próprio vetor de entrada, pois a posição de escrita fica sempre atrás da de leitura.

- **Exercício 5:** um firewall registrou n endereços IPv4 (milhões de entradas), e a equipe de segurança quer a lista em ordem numérica para detectar blocos contíguos suspeitos. Cada endereço é formado por 4 bytes (ex.: 10.2.200.1). Qual é a forma mais eficiente que você conhece para solucionar esse problema?
- **Ideia:** interpretar cada endereço como um número de $w = 4$ dígitos em base $k = 256$ (cada byte é um dígito), considerando o byte mais à esquerda como o mais significativo. Aplicar radix sort: 4 passadas de counting sort, uma por byte, do menos significativo (byte 3) para o mais significativo (byte 0).

- **Exercício 5:** um firewall registrou n endereços IPv4 (milhões de entradas), e a equipe de segurança quer a lista em ordem numérica para detectar blocos contíguos suspeitos. Cada endereço é formado por 4 bytes (ex.: 10.2.200.1). Qual é a forma mais eficiente que você conhece para solucionar esse problema?

```
sort_ips(v[], n):                // v[i] = 4-byte address
    for w = 3 down to 0:
        for j = 0 to 256: fp[j] = 0
        for i = 0 to n-1:
            fp[byte(v[i], w) + 1] += 1
        for j = 1 to 256:
            fp[j] += fp[j-1]
        for i = 0 to n-1:
            aux[fp[byte(v[i], w)]] = v[i]
            fp[byte(v[i], w)] += 1
        for i = 0 to n-1:
            v[i] = aux[i]
```

- **Exercício 5:** um firewall registrou n endereços IPv4 (milhões de entradas), e a equipe de segurança quer a lista em ordem numérica para detectar blocos contíguos suspeitos. Cada endereço é formado por 4 bytes (ex.: 10.2.200.1). Qual é a forma mais eficiente que você conhece para solucionar esse problema?
- **Análise:** cada passada custa $\Theta(n + k)$ com $k = 256$; com $w = 4$ passadas, $\Theta(w(n + k)) = \Theta(n)$, já que w e k são constantes. Espaço adicional: $O(n + k)$ para o auxiliar e os contadores. O tamanho do dígito é um parâmetro de projeto: dígitos maiores reduzem w mas aumentam k exponencialmente (b bits $\Rightarrow k = 2^b$); enquanto k não ultrapassa a ordem de n , o custo segue $\Theta(n)$. O byte é a escolha usual por conveniência, não por ser o ótimo.

Perguntas?

`thiago.p.araujo@fgv.br`